

Research prototype - simulated or research data.

AirportXR Companion Long-Haul Layover and Inter-Airport Transit Report

Research prototype - simulated or research data.

Report date: 2026-07-14

Canonical source: Documentation/Reports/2026-07-14-airportxr-long-haul-layover.md

Project: AirportXRCompanion

Scope: Long-haul journey support, same-airport layovers, inter-airport transfers, interval activities, entry guidance, flight offers and connection baggage, delay and capacity evidence, cross-device reminders, optional AI explanation, data provenance, safety classification, privacy, validation, and design rationale.

Evidence cutoff: Source, documentation, tests, and logs observed on 2026-07-14. Each validation result below identifies its own observation time and source boundary; an earlier result does not prove later edits.

Repository state: No Git commit identifier was recorded for this working snapshot. File paths and SHA-256 hashes are therefore the reproducibility anchors.

Executive summary

AirportXR Companion is a native iPhone research prototype for travelers managing complex long-haul journeys. Its central interaction is not a list of attractions. It is a conservative answer to a more important question: what should the traveler do next, given the remaining time, the uncertainty in each required step, the freshness and authority of the data, and the consequences of missing the onward flight?

The current source supports three itinerary classes:

- Same-airport connection, such as BKK-HND-LAX.
- Same-region inter-airport connection, such as BKK-HND, a surface transfer from HND to NRT, and then NRT-LAX.
- Offline or partially known journeys in which the app keeps the itinerary visible but withholds a safety claim until critical inputs are confirmed.

The recommendation engine is deterministic for a fixed versioned input snapshot. It uses 10,000 Monte Carlo trials, triangular duration distributions, a stable seed, and a Wilson 95 percent confidence interval. The classification is deliberately asymmetric: the app requires strong evidence before saying SAFE, identifies clearly poor candidates as NOT RECOMMENDED, and labels the uncertainty between those bounds TIGHT. Any unresolved critical input preempts those labels and produces REQUIRES CONFIRMATION.

The inter-airport design treats the HND-to-NRT movement as a mandatory itinerary segment. It ranks current, accessible transfer options on duration, transfers, and walking without hiding those tradeoffs inside one opaque score. It can then consider an optional activity near the onward airport only after the transfer and required airport processes fit inside the available interval. Demo estimates are explicitly labeled and cannot support a live fastest-route claim.

The live/demo boundary is enforced through dependency selection, not only copy. Demo and stochastic modes may create named itinerary, profile, candidate, and HND-to-NRT fixtures. Live and offline modes load protected cached facts or remain unavailable; they do not synthesize the reference itinerary, candidate durations, or Tokyo demo transfer. Entry queries also remain blocked until traveler facts, sourced airport countries, and itinerary times are complete.

The product design is informed by primary-source patterns from Apple, Flighty, Fly Delta, Triplt, airport operators, and transit standards. The resulting design keeps the next action prominent, maintains one continuous itinerary, uses familiar airport-signage structure, explains data freshness in words, and reveals the calculation trace progressively. Every major design choice and its rationale is documented below.

The repository also contains four deliberately bounded foundations: an exact-query entry-provider/cache chain; a provider-neutral delay, commercial-capacity, external flight-offer, and connection-baggage layer with a verified Flights Book & bags surface; opt-in Apple Calendar and Google Tasks reminder adapters; and a constrained Workers AI explanation route. These are not equivalent to complete live-provider integration. Book & bags uses only exact-match DEMO previews or truthful live/offline unavailability, the AI route has no iOS affordance, and Google Tasks remains visibly disabled until a real OAuth client exists. No in-app payment, ticket issuance, visa decision, overbooking guarantee, bin-space guarantee, or AI safety decision is implemented.

The synchronized local acceptance snapshot passed a clean terminal build, 78 iOS unit tests, 13 UI tests, 37 mocked Worker tests, the Worker static check, and a Wrangler dry run. Ordinary non-test simulator launches were visually reviewed in demo, offline, stochastic, live-requested, and HND-to-NRT inter-airport modes. The live-requested screen correctly remained on named fixture data because no proxy configuration or provider credentials were installed. No live commercial response, cross-device account delivery, live model inference, production deployment, or physical-device AR result was observed or is claimed.

Deterministic safety policy - first technical table

This is the first technical table in the report because the safety contract governs every later feature and screen.

Policy element	Deterministic value	Rationale and boundary behavior	Source
Policy version	layover-safety-2026-07-14-v1	Versioning makes a recommendation reproducible and prevents silent threshold drift.	AirportXRCompanion/Services/LongHaul/SafetyPolicy.swift
Confidence level	0.95	A 95 percent interval expresses simulation uncertainty without presenting the point estimate as certainty.	AirportXRCompanion/Services/LongHaul/SafetyPolicy.swift
Normal critical value	1.959963984540054	Fixed z value makes the Wilson interval deterministic across runs.	AirportXRCompanion/Services/LongHaul/SafetyPolicy.swift
SAFE threshold	Wilson lower bound ≥ 0.90	Even the conservative end of the interval must meet 90 percent. Equality at 0.90 is SAFE.	AirportXRCompanion/Services/LongHaul/SafetyPolicy.swift
NOT RECOMMENDED threshold	Wilson upper bound < 0.70	Even the optimistic end is below 70 percent. Equality at 0.70 is not enough for this label and remains TIGHT.	AirportXRCompanion/Services/LongHaul/SafetyPolicy.swift
TIGHT region	All other resolved intervals	This middle state communicates material uncertainty without a false binary answer.	AirportXRCompanion/Services/LongHaul/SafetyPolicy.swift
Critical-input preemption	Any unresolved critical input -> REQUIRES CONFIRMATION	Missing border, check-in, security, transfer, terminal, or other required timing evidence must block a numerical safety claim.	AirportXRCompanion/Services/LongHaul/MonteCarloLayoverRecommendationEngine.swift
Target maximum half-width	$e = 0.01$	A one-percentage-point planning target produces a stable, explainable simulation budget.	AirportXRCompanion/Services/LongHaul/SafetyPolicy.swift

Policy element	Deterministic value	Rationale and boundary behavior	Source
Raw conservative trial count	$\text{ceil}(z^2 \times 0.25 / e^2) = \text{ceil}(9603.647051735314) = 9604$	The 0.25 Bernoulli variance is the worst case, so the count does not depend on an optimistic prior.	AirportXRCompanion/Services/LongHaul/SafetyPolicy.swift
Production trial count	10,000	The policy rounds 9,604 upward to a memorable value; it never rounds below the calculated requirement.	AirportXRCompanion/Services/LongHaul/SafetyPolicy.swift
Stable seed	SHA-256 of itinerary UUID, input revision, snapshot revision, and policy version; UInt64 prefix	A fixed evidence snapshot produces the same simulated sequence and recommendation trace.	AirportXRCompanion/Services/LongHaul/SafetyPolicy.swift
Segment distribution	Independent triangular distribution per duration segment	Minimum, likely, and maximum values are understandable and usable when full empirical distributions are unavailable. Independence is a documented limitation.	AirportXRCompanion/Services/LongHaul/MonteCarloLayoverRecommendationEngine.swift
Available-time definition	Gate-close time minus inbound on-block time	Gate close, rather than scheduled departure, is the relevant deadline for the modeled traveler action.	AirportXRCompanion/Services/LongHaul/MonteCarloLayoverRecommendationEngine.swift

The 0.90 and 0.70 boundaries are provisional product-governance choices from the approved research plan. They are not returned by a flight, weather, airport, immigration, or AI API; they are not a certified aviation standard; and the repository does not contain outcome evidence proving that they are optimal. They are deterministic because a fixed evidence snapshot must be replayable and auditable, not because the numbers should remain immutable forever.

A randomly changing threshold is not safer. It could give different advice for the same traveler, itinerary, and source snapshot, obscure regressions, and make an offline result impossible to explain. The preferred evolution is contextual but still deterministic: derive a versioned threshold from reviewed consequence classes, prediction calibration, separate-ticket exposure, overnight risk, accessibility impact, and other declared inputs, then apply a non-weakenable product safety floor. Traveler preference may make a plan more conservative but must never lower that floor. Any change requires a new policy version, rationale, calibration/backtest evidence, boundary tests, and a migration rule. Live launches remain naturally variable because their evidence, clock, network, location, cache, and learned state change; the decision rule itself should remain stable for one snapshot.

Formula and classification

For trial i :

CODE - TEXT

```

required_i = deplane_i
            + border_i
            + baggage_i
            + terminal_or_surface_transfer_i
            + check_in_i
            + security_i
            + gate_walk_i
            + candidate_activity_i
            + policy_buffer_i

available_i = onward_gate_close - inbound_on_block
success_i = 1 when required_i <= available_i, otherwise 0
p_hat = sum(success_i) / n

```

For $n = 10,000$ and $z = 1.959963984540054$, the Wilson bounds are:

CODE - TEXT

```

denominator = 1 + z^2 / n
center = (p_hat + z^2 / (2n)) / denominator
margin = z * sqrt((p_hat * (1 - p_hat) / n) + z^2 / (4n^2))
        / denominator
lower_95 = center - margin
upper_95 = center + margin

```

The decision order matters:

1. If a required critical input is missing, expired, or unresolved, return REQUIRES CONFIRMATION and do not show a probability-based approval.
2. Otherwise, if $\text{lower_95} \geq 0.90$, return SAFE.
3. Otherwise, if $\text{upper_95} < 0.70$, return NOT RECOMMENDED.
4. Otherwise, return TIGHT.

The point estimate, interval, trial count, seed provenance, policy version, data freshness, and unresolved fields belong in the calculation trace. They are decision-support evidence, not a guarantee of connection success.

Removal of hidden operational constants

Deterministic does not mean inventing universal operational minutes. The legacy risk helpers were reconciled so their inputs and classifications expose the source of time:

- BoardingRiskService no longer derives boarding as departure minus 30 minutes. Without airline/provider boarding time it returns unavailable and asks for verification.
- Flight freshness uses source/provider policy rather than a universal age inside the decision layer.
- Leave soon is derived from the current route plus sourced route uncertainty, not a fixed two- or five-minute localization adjustment.
- ConnectionRiskService no longer adds an eight-minute terminal-change penalty or a fixed 15-minute tight band. Callers supply terminal/inter-airport movement, walking, and accessibility impacts; the recovery window scales with that context.
- LayoverSafetyService no longer uses a fixed 45-minute band. It classifies the remaining margin after every supplied process and safety component.

Named demo and test fixtures may still contain explicit numbers so they can be replayed. Those values remain provenance-labeled and cannot cross into live mode.

Scope, method, and evidence rules

Questions addressed

This report evaluates:

- What the current prototype implements for long-haul layovers.
- How the HND-to-NRT case should behave when two airports serve the same metro region.
- How optional interval activities can be offered without displacing the mandatory airport transfer.
- How flight offers, baggage reclaim/recheck, separate-ticket self-transfer, and inter-airport movement feed connection risk.
- How weather, congestion, and rotation features can support a delay distribution without replacing official flight status.
- What can and cannot be inferred about overbooking, seat availability, checked-bag acceptance, and overhead-bin space.
- How mutable entry rules use structured providers, protected cache, official links, and a non-authoritative search/AI fallback.
- Which exact derived times may create cross-device reminders and which consent/runtime boundaries still remain.
- How optional AI can explain sourced facts without changing feasibility or entry decisions.
- Which design patterns are established in relevant travel and iOS products.
- Why each major product and visual design choice was made.
- How live, cached, stale, demo, and unavailable data are distinguished.
- Where provider keys belong and what the app does when an API fails.
- What privacy and security constraints apply.
- What has and has not been validated.

Evidence classes

Observed source evidence means the behavior is visible in the current repository source. Observed log evidence means a saved log records the result at a stated time. Primary-source research means the claim is tied directly to an operator, platform owner, or standard owner. Demo evidence means a fixture exists for repeatable research and must not be interpreted as current operational data. Planned behavior means the architecture or interface exists but current end-to-end validation is incomplete.

The report does not infer live status from a demo fixture, does not turn a published static duration range into a current fastest-route claim, and does not treat the presence of a provider adapter as proof that credentials were configured.

Canonical implementation and documentation inputs

The principal evidence files are:

- .codex/project-context.md
- Documentation/UIUXReview.md
- Documentation/Architecture.md

- Documentation/DataSources.md
- Documentation/LiveDataPlan.md
- Documentation/PrivacyAndSecurity.md
- Documentation/SecurityReview.md
- Documentation/AllIntegrations.md
- Documentation/CrossDeviceReminders.md
- Documentation/FlightDelayCapacityAndBookingArchitecture.md
- Documentation/UserDemoGuide.md
- Documentation/DeveloperGuide.md
- AirportXRCompanion/Services/LongHaul/SafetyPolicy.swift
- AirportXRCompanion/Services/LongHaul/MonteCarloLayoverRecommendationEngine.swift
- AirportXRCompanion/Services/LongHaul/LongHaulReferenceScenario.swift
- AirportXRCompanion/Models/LongHaul/InterAirportTransferModels.swift
- AirportXRCompanion/Models/LongHaul/MetroAirportDatabase.swift
- AirportXRCompanion/ViewModels/LongHaulExperienceViewModel.swift
- AirportXRCompanion/Views/Transit/TransitView.swift
- AirportXRCompanion/Components/LongHaulInterfaceComponents.swift
- AirportXRCompanion/Aviation/Repositories/FlightRepository.swift
- AirportXRCompanion/Aviation/Caching/FlightCache.swift
- AirportXRCompanion/Persistence/ItineraryCache.swift
- AirportXRCompanion/Services/OnDevicePredictionService.swift
- AirportXRCompanion/Models/FlightOperationalIntelligenceModels.swift
- AirportXRCompanion/Services/FlightOperationalIntelligenceService.swift
- AirportXRCompanion/Views/Flights/FlightsView.swift
- AirportXRCompanion/Reminders/CrossDeviceReminderModels.swift
- AirportXRCompanion/Reminders/CrossDeviceReminderPlanner.swift
- AirportXRCompanion/Reminders/CrossDeviceReminderCoordinator.swift
- AirportXRCompanion/Reminders/AppleCalendarReminderAdapter.swift
- AirportXRCompanion/Reminders/GoogleTasksProxyReminderAdapter.swift
- AirportXRCompanion/Persistence/EntryRequirementCache.swift
- AirportXRCompanion/Aviation/Providers/ProxyFlightDataProvider.swift
- AirportXRCompanion/Aviation/Networking/FlightAPIError.swift
- AirportXRCompanion/Resources/PrivacyInfo.xcprivacy
- AirportXRCompanion/App/AppContainer.swift
- AirportXRCompanion/App/RootTabView.swift

- AirportXRCompanion/Configuration/AppLaunchContext.swift
- AirportXRCompanionTests/LongHaulExpansionTests.swift
- AirportXRCompanionTests/FlightDataTests.swift
- AirportXRCompanionTests/TransitNotificationPredictionTests.swift
- AirportXRCompanionTests/CrossDeviceReminderTests.swift
- AirportXRCompanionTests/FlightOperationalIntelligenceTests.swift
- AirportXRCompanionUITests/AirportXRCompanionUITests.swift
- AirportXRCompanionUITests/FlightBookAndBagsUITests.swift
- build/current-unit-tests-final.log
- build/current-build-for-testing-final.log
- build/current-backend-tests.log
- build/ui-accessibility-rerun.log
- Backend/src/index.js
- Backend/src/ai-explainer.js
- Backend/test/worker.test.js
- Backend/README.md
- Backend/wrangler.toml

Implemented product capabilities

One continuous journey

The root application exposes six stable native destinations:

1. Journey
2. Flights
3. AR Guide
4. Map
5. Layover & Transit
6. Settings

AirportXRCompanion/App/RootTabView.swift is the implementation source. The stable root structure preserves orientation while the Journey screen carries the time-sensitive next action. This avoids forcing the traveler to rediscover features during a high-stress connection.

Flight and itinerary state

The application can represent scheduled and operational flight fields, keep a cached itinerary available, label freshness, and apply configured provider fallback. AirportXRCompanion/Aviation/Repositories/FlightRepository.swift owns flight-search provider order and may return an explicitly labeled demo result when AppContainer includes DemoFlightDataProvider because demo fallback is enabled or no live flight provider exists. The long-haul experience has a stricter boundary: live and offline launches load only the cached itinerary or remain empty, and never synthesize the BKK-HND-LAX reference itinerary or same-airport demo candidates. AirportXRCompanion/Aviation/Caching/FlightCache.swift and AirportXRCompanion/Persistence/ItineraryCache.swift provide protected local persistence.

The 2026-07-16 itinerary revision adopts a boarding-pass hierarchy supported by Apple Wallet and current airline day-of-travel products. Each leg presents flight/status, departure and arrival airport codes, separate local gate times and short dates, city/terminal/gate, an explicit date-line day offset, then boarding time/gate/group. This replaces the earlier combined abbreviated date at time -> abbreviated date at time sentence, which was visually long and made local-time ownership hard to scan. Journey and Flights share the same AirlineFlightSummary component so operational information does not change shape between viewing and editing.

Indoor position and AR replay

The simulator does not obtain a real indoor location. Explicit ar-preview and walkthrough launches use TerminalRouteLocationEmulator, which interpolates repeatable local map points and floors along the selected graph and updates the maneuver card as landmarks are reached. Normal launches never enable this replay. On physical iPhone hardware, ARKit supplies camera-relative world tracking; the current implementation does not claim that this establishes a gate or corridor coordinate.

Parallel signal testing is provided by Scripts/indoor-signal-emulator.py. This process owns its clock independently of the iOS app and supports pause, resume, next, previous, and reset controls. The ar-external launch mode polls loopback JSON. The ar-corelocation mode instead sends each projected coordinate with simctl location; Apple Simulator delivers it through CLLocationManager, and HTTP remains only the process-control channel. HNDCoreLocationQAFixture is a versioned synthetic transform around the HND reference point, not surveyed indoor geometry; its displayed floor is obtained from the matched test-graph node because latitude/longitude does not establish a terminal level. Both paths are rejected outside explicit UI-test mode.

AR instructions now classify signed graph-edge angles as straight, slight, standard, sharp, or U-turn and derive an eight-way outgoing compass heading. This supports short passenger wording such as "Continue northwest," "Bear left," and "Make a sharp right." A single semantic maneuver drives the text, accessibility label, route-step list, SF Symbol, and RealityKit arrow rotation. The angular bands are centralized in RouteTurnPresentationPolicy and affect presentation only; they do not alter the selected route or its distance.

Production indoor placement should use venue-owned IMDF and indoor positioning where available, with optional beacon proximity, UWB accessories, or surveyed visual relocalization as partner integrations. Core Location replay remains directly suitable for outdoor airport approaches and HND-NRT surface movement. Indoor QA requires the explicit synthetic transform described above; production latitude/longitude alone cannot establish a floor or corridor. The provider boundary and research matrix are recorded in Documentation/IndoorPositioningArchitecture.md.

Long-haul layover recommendations

The engine models mandatory airport processes and optional candidates as uncertain durations rather than exact promises. It returns:

- SAFE when conservative evidence clears the policy threshold.
- TIGHT when a candidate is plausible but uncertain.
- NOT RECOMMENDED when even the optimistic interval is below policy.

- REQUIRES CONFIRMATION when critical data is missing or stale.

The calculation trace records why the label was produced. This makes the result inspectable and testable.

Same-region inter-airport transfer

When the inbound and outbound airport codes differ, the itinerary is not treated as a normal terminal change. The current source:

- Classifies same-airport, same-metro, and cross-region surface movements.
- Finds candidate airports through a versioned metro-airport database.
- Obtains route estimates from an injected transfer provider.
- Rejects routes that are not current enough for a live claim.
- Applies accessibility requirements when provided.
- Computes the Pareto frontier over duration, number of transfers, and walking.
- Uses deterministic lexicographic tie-breaking.
- Allows canClaimFastest only when the selected route is LIVE and no route input remains unresolved; a ranked DEMO route always returns false.
- Inserts the surface transfer before any optional activity.
- Withholds the arrival target if border, check-in, security, terminal-route, or safety distributions are unresolved.

The core models and planner are in `AirportXRCompanion/Models/LongHaul/InterAirportTransferModels.swift`. The demo provider and candidate factory are in `AirportXRCompanion/Services/LongHaul/LongHaulReferenceScenario.swift`. Presentation behavior is in `AirportXRCompanion/ViewModels/LongHaulExperienceViewModel.swift` and `AirportXRCompanion/Views/Transit/TransitView.swift`.

Facilities and interval activities

The reference facility registry supports airport work areas, sleep, showers, lounges, observation, dining, and nearby discovery with source and access-zone metadata. The app distinguishes:

- Facility existence from current availability.
- Airside from landside access.
- Eligibility from general public access.
- Official source evidence from third-party discovery.
- An optional activity from a mandatory transfer step.

For HND, the current registry correctly treats the Terminal 3 work pod as general or landside, not airside. The transit hotel is airside, while the Terminal 3 shower is in the landside arrival lobby. Availability and eligibility still require confirmation.

Entry, border, and baggage checks

Entry eligibility is a critical input when an activity or transfer may require entering the country. The current normalized Worker chain is:

exact itinerary and minimal traveler query -> Sherpa v3 structured trip guidance -> independently contracted Timatic normalized adapter on compatible failure -> optional Gemini Google Search official-link discovery -> bundled allowlisted official links

The Worker rejects passport numbers, document numbers, scans, images, and MRZ content. Authentication and rate-limit failures do not trigger a fallback storm. A structured response retains its provider chain, record ID, observed/received/expiry times, requirements, and official verification links. It remains guidance: the current Worker returns REQUIRES CONFIRMATION rather than promising that the traveler may enter.

Gemini is not a backup visa decision API. Its request contains a destination country and deployment-allowlisted official-government domains. Generated prose is discarded; only HTTPS citations on those domains survive. The normalized evidence kind is officialSourceDiscovery, it returns no derived requirements, and it can never support a positive landside or city recommendation. When neither structured provider is configured, the bundled Japanese Ministry of Foreign Affairs link is an immediately expired informational pointer, not a rule engine.

AirportXRCompanion/Persistence/EntryRequirementCache.swift protects the normalized response under a one-way SHA-256 fingerprint of the complete query, including nationality, residence, passport type, declared authorizations, airports, countries, exact dates, time zones, landside intent, luggage, and purpose. The cache is versioned by provider policy. A matching unexpired structured record may be reused; an expired record remains display/provenance evidence only; a different traveler or itinerary cannot collide through a broad country-only key. A transient informational result does not overwrite a current structured record. Offline mode returns only an exact cache match or remains unavailable.

Baggage through-check is also represented as a connection-specific evidence item. The UI and domain must not infer entry permission or through-check from nationality, flight number, airport pairing, matching carriers, or alliance membership alone.

Live and offline modes begin with TravelerProfile.incomplete unless a protected cached profile exists. An entry request is not constructed until nationality and residence are present, the transit and onward airport country codes resolve through the versioned airport reference registry, and the relevant arrival and departure times exist. Demo and stochastic modes may use the named minimal demo profile so fixture behavior remains reproducible.

Flight delay, commercial capacity, offers, and connection baggage

AirportXRCompanion/Models/FlightOperationalIntelligenceModels.swift and AirportXRCompanion/Services/FlightOperationalIntelligenceService.swift implement provider-neutral contracts and validation services. AirportXRCompanion/Views/Flights/FlightsView.swift exposes their current user-facing boundary in a third Book & bags mode. This is not a claim that a live airline, weather, congestion, NDC, order, or booking adapter is currently configured in AppContainer.

The delay layer accepts separately sourced weather, airport-congestion, inbound-rotation, and schedule features. Each feature retains physical unit, provider field, source record, observation/receipt/expiry times, uncertainty, and derivation steps. A versioned model may return ordered arrival-delay quantiles and optional exceedance probabilities. The service rejects an absent model, wrong flight identity, missing or stale required features, crossing quantiles, invalid probabilities, missing declared inputs, or expired output. Official flight status and effective airline times remain authoritative; an optional prediction is scenario context and never overwrites them. A departure-only model is not silently reused as an arrival model.

The commercial-fact normalizer keeps evidence classes distinct:

- A current airline order or PNR may support that ticketed traveler's baggage allowance or confirmed reservation.
- A current operating-airline control fact is required to assert that the flight is or is not oversold, or that bin space is guaranteed.
- A seat map, fare inventory, shopping offer, boarding group, or third-party estimate remains a proxy.
- Missing evidence yields UNKNOWN, not a favorable assumption.

No verified universal public consumer API was found for guaranteed overbooking status or remaining overhead-bin space. The correct production fallback is unknown plus operating-airline verification.

The flight-offer layer can validate a current airline/NDC priced offer, retain exact decimal price/currency, ordered legs, baggage quote, source, and expiry, and request an identity-matched HTTPS external handoff. It intentionally has no payment or ticket-issuance protocol. The provider or airline confirms price, fulfillment, ticketing, and post-booking service after handoff.

For every connection, `ConnectionBaggageState` distinguishes throughChecked, reclaimImmigrationCustomsRecheck, selfTransferSeparateTicket, confirmationRequired, and unknown. Only a current airline operations/order/PNR/airline-offer/NDC-offer record can assert throughChecked; carrier matching is insufficient. `ConnectionTransferPlanBuilder` then creates only the sourced segments required by that state:

baggage wait

- border / immigration when required
- customs when required
- landside movement
- bag drop / check-in, constrained by its sourced cutoff
- security
- inter-airport travel when airport codes differ

No duration is generated by the builder. HND-to-NRT automatically requires a separately sourced inter-airport segment. Missing or stale required duration evidence, or a missing bag-drop cutoff when recheck is needed, remains unresolved and prevents a positive connection recommendation. Each result maps into existing `PlanSegment` inputs so baggage handling and predicted arrival uncertainty can affect connection risk without bypassing the safety engine.

Book & bags renders one connection card between each pair of adjacent itinerary legs. The named BKK-HND-LAX demo says the checked bag is through-checked to LAX from a seeded provider record. The HND-to-NRT demo says reclaim, immigration, customs, landside movement, recheck, security, inter-airport travel, and a sourced bag-drop cutoff are required. Each card shows the exact state label, DEMO/data mode, verification/provenance, required segments and distributions, cutoff, and unresolved facts. Live and offline modes remain UNKNOWN when no provider evidence exists.

The offer form accepts route, departure date, and adult count. It returns the seeded demo offer only for an exact fixture match and labels it NOT LIVE OR BOOKABLE. A mismatch does not query or invent a different offer. Live/offline mode reports provider unavailable, and the only continuation is the validated HTTPS external handoff. The screen and its two dedicated UI cases are included in the passing 13-test UI acceptance artifact. That result validates fixture, unknown-state, and handoff behavior; it does not create a live offer or airline order.

Cross-device reminders

`CrossDeviceReminderPlanner` creates only three kinds of stable intent from already-derived future facts: Go to Gate from `JourneyAssessment.leaveBy`, sourced gate close, and latest return from a selected SAFE plan with no unresolved trace inputs. There is no hidden fixed lead-time constant. Missing, expired, unsafe, unresolved, or past inputs create no reminder.

`AppleCalendarReminderAdapter` is explicit opt-in, requests full Calendar access only from the user's enable action, and upserts or deletes only events carrying the app's stable airportxr reminder marker. The event and alarm use the exact already-derived action time. A one-minute event length is only a versioned `EventKit` action marker, not a travel estimate. Cross-device appearance depends on the calendar account the user selects and is not guaranteed by `EventKit` alone.

GoogleTasksProxyReminderAdapter uses a short-lived user OAuth token, never an API key, and the protected Worker route performs a bounded idempotency scan before mutation. Stable markers make retries update instead of duplicate. Google Tasks retains only a due date through its API; the exact action instant and airport time zone are written in notes while Google controls reminder timing. Apple Calendar is therefore the exact-time path in this design.

The planner, adapters, Worker route, consent model, and Apple Calendar AppContainer/Settings runtime source exist. Apple sync is default-off; only an explicit toggle action may request full EventKit access. Versioned consent, itinerary scope, last-sync state, active-app refresh, stale-scope cleanup, opt-out cleanup, and error/status UI are implemented in source. Demo and stochastic modes cannot write calendar events. Gate-close export requires ordered timestamps and an explicit future expiry; latest-return additionally requires a selected SAFE/no-unresolved assessment, current on-block/gate-close and every required segment, current landside entry when relevant, and no unproven last-service timestamp. The complete application build and automated unit/UI suites passed. Cross-device account delivery still requires a separately configured physical-device/account exercise.

Google Tasks is shown disabled with OAuth setup required; no fake token provider is injected. Google OAuth client integration, continuously revising reminders while the app is suspended, and two-device account delivery remain separate production acceptance surfaces.

Optional grounded AI explanation

POST /v1/ai/explain is an optional backend-only explanation route. It accepts either a sourced CalculationTrace or sourced facility records. The model sees stable fact identifiers and short descriptors, not traveler profiles, operational values, provider record IDs, or arbitrary prompts. It may only order every supplied fact identifier exactly once and select a focus enum. Deterministic Worker code renders all values, formulas, timestamps, caveats, provenance, hours, and official URLs. A missing, duplicate, invented, or malformed ID falls back to the original deterministic order; raw model prose is never returned.

Entry, visa, passport, nationality, citizenship, and immigration content is rejected before inference. AI cannot alter feasibility, select a safety threshold, authorize entry, create an availability claim, or authorize native-model training. Responses are no-store. The current implementation uses a Cloudflare Workers AI binding and therefore places no AI key in the iOS app. There is no current iOS explanation button or renderer, and deterministic traces remain the fallback when the binding, quota, or model is unavailable.

Free and trial access checked in the project research on 2026-07-14 is changeable and must be reverified at deployment:

AI option	Free/trial posture at research cutoff	Recommended role and limitation
Cloudflare Workers AI	Cloudflare documents 10,000 neurons per day at no charge on Free and Paid Workers plans, with model-specific use and a daily reset	Implemented server option because the Worker already owns authentication and rate limits. Enforce a smaller product budget and fall back to deterministic order when quota is exhausted.
Google Gemini Developer API	Selected models may have a free tier, but model/project rate limits change; Google Search grounding used by the entry adapter is not assumed to be free	Official-government link discovery only. Unpaid-service data terms make traveler profiles/documents inappropriate; prose is discarded and no eligibility is inferred.
Hugging Face Inference Providers	Free Hub accounts currently receive a small USD 0.10 monthly credit, subject to change	Useful for experiments and portability, not enough for a primary passenger path; model, routed provider, token scope, and license require separate review.
Apple Foundation Models	Uses the on-device Apple Intelligence model rather than a metered web API on supported devices/regions	Future private/offline explanation option. It is not implemented, is unavailable on part of the iOS 18 device base, and is not a world-knowledge or operational-data authority.

Advisory OCR

Apple Vision can recognize text locally from a user-selected image. Any optional cloud OCR path is default-off and requires explicit consent for one cropped, re-encoded still image. OCR can help populate fields, but recognized text remains advisory until the user confirms it.

Data state and accessibility

Shared UI components label LIVE, CACHED, STALE, DEMO, and UNAVAILABLE conditions. Status is expressed with words and symbols as well as color. The layout uses semantic text and surfaces and is designed to adapt to Dynamic Type rather than freezing information in a dense aviation dashboard.

Reference scenario A - BKK to HND to LAX

AirportXRCompanion/Services/LongHaul/LongHaulReferenceScenario.swift contains a named, repeatable demo fixture:

- Scenario version: demo-bkk-hnd-lax-2026-07-14-v1
- Inbound: TG660, BKK to HND
- Onward: NH106, HND to LAX
- Layover: 370 minutes in the fixture
- Airport time zone: Asia/Tokyo
- Data authority: demo fixture, not a current flight status claim

The scenario exercises a same-airport connection. A valid candidate sequence remains inside HND unless entry and landside return steps are explicitly modeled. The app can compare rest, work, shower, lounge, observation, and dining candidates against required border, terminal, security, walking, and policy buffers.

The value of the fixture is reproducibility. It does not establish that TG660 or NH106 is operating at the fixture times on any current date.

Reference scenario B - BKK to HND, HND to NRT, NRT to LAX

Scenario structure

The inter-airport demo in AirportXRCompanion/Services/LongHaul/LongHaulReferenceScenario.swift represents:

- Inbound: TG660, BKK to HND
- Mandatory surface segment: HND to NRT
- Onward: NH6, NRT to LAX
- Demo onward departure: nine hours after the scenario anchor
- Demo gate-close rule: 40 minutes before departure
- Baggage state: confirm through-check
- Data authority: demo fixture, not a live itinerary or route claim

This scenario is intentionally harder than a same-airport layover. It can require immigration, baggage handling, ground transport, check-in, security, terminal navigation, and a gate walk. A missing input in any critical step prevents a safety recommendation.

FlightOperationalReferenceFixtures.hndToNrt(anchor:seed:) adds a separate seeded, DEMO-labeled operational-intelligence fixture for the Book & bags screen and tests. It contains a reclaim/immigration/customs/recheck connection state, sourced process distributions, a bag-drop cutoff, delay-feature/model output, capacity evidence, and a priced external offer. The same seed replays exactly, and every fact remains demo provenance. Its screen is build- and UI-test-verified, but does not establish a live offer, airline order, route, delay, overbooking state, or luggage-capacity claim.

Demo route estimates

The TokyoInterAirportDemoTransferProvider supplies the following triangular research ranges:

Mode	Minimum	Likely	Maximum	Walking	Transfers	Accessibility	Authority
Rail	82 min	96 min	125 min	420 m	0	Supported	Demo
Airport bus	70 min	100 min	145 min	180 m	0	Unknown	Demo
Road	60 min	92 min	150 min	90 m	0	Unknown	Demo

These values let tests and prototypes exercise ranking and uncertainty. They must be shown as DEMO - NOT LIVE. The likely demo duration happens to be lowest for road at 92 minutes, but that does not prove road is currently fastest. Traffic, waiting time, missed departures, service disruption, terminal origin, terminal destination, accessibility, luggage, and provider freshness can reverse the ordering.

Haneda Airport's official static access page publishes example HND-to-NRT ranges of approximately 90 to 115 minutes by train and 65 to 85 minutes by airport shuttle bus, while warning that times and fares can change. Those published ranges are useful for contextual review, but they are not a current journey plan. Source: <https://tokyo-haneda.com/en/access/narita/index.html>

How the fastest suitable route should be selected

The app should claim a fastest suitable route only when:

1. The route response is current and its timestamp is visible.
2. Origin and destination terminals are known or conservatively represented.
3. Wait time and the next usable departure are included, not just in-vehicle time.
4. Known disruption and last-service conditions have been applied.
5. The route satisfies the traveler's stated accessibility constraints.
6. The full range, not only the optimistic duration, fits the connection policy.
7. A fallback option is shown in case the leading route fails.

The planner first removes dominated options. A route is dominated when another is no worse on duration, transfers, and walking and is better on at least one. It then applies a stable lexicographic order: duration, transfers, walking, then provider identifier. This is intentionally not a hidden scalar score. The traveler can see why one route leads.

Until a live GTFS, GTFS-Realtime, or licensed regional journey-planning adapter supplies current results, the UI should say Leading demo option - not live, not Fastest route. InterAirportTransferPlan.canClaimFastest now enforces this at the domain boundary: it is true only for a selected LIVE route with no unresolved input, so every DEMO route returns false even when it ranks first.

Arrival target and interval activities

The traveler needs a target arrival time at NRT, not only a departure instruction from HND. The source-derived arrival target is shown only when the critical distributions for border, check-in, security, terminal routing, and policy buffer are known. Otherwise the target reads as unknown and the card requests confirmation.

Optional activity logic follows this order:

Land at HND -> complete required border and baggage steps -> travel HND to NRT -> establish safe NRT arrival and onward processing -> consider a nearby NRT-area activity -> return to the onward gate plan

The optional location must never be inserted between HND arrival and the mandatory transfer merely because it is geographically nearby. A candidate is eligible only when the simulation includes:

- Exit or entry requirements.
- Baggage state and storage, if needed.
- The complete transfer range.
- NRT check-in or document check.
- Security and terminal movement.
- Opening hours and access zone.
- Travel to and from the activity.
- A policy buffer.

If those inputs are incomplete, the app can still show places as discovery information, but it must not recommend visiting them during the interval.

Metro-airport database

AirportXRCompanion/Models/LongHaul/MetroAirportDatabase.swift defines version metro-airports-2026-07-14-v1. It contains 25 curated multi-airport metro records:

- Tokyo: HND and NRT
- Osaka: KIX, ITM, and UKB
- Seoul: ICN and GMP
- Bangkok: BKK and DMK
- Beijing: PEK and PKX
- Shanghai: PVG and SHA
- Taipei: TPE and TSA
- Kuala Lumpur: KUL and SZB
- Istanbul: IST and SAW
- London: LHR, LGW, STN, LTN, LCY, and SEN
- Paris: CDG, ORY, and BVA
- Milan: MXP, LIN, and BGY
- Rome: FCO and CIA
- Stockholm: ARN, BMA, NYO, and VST

- New York: JFK, LGA, and EWR
- Washington-Baltimore: DCA, IAD, and BWI
- Chicago: ORD and MDW
- San Francisco Bay Area: SFO, OAK, and SJC
- Los Angeles: LAX, BUR, LGB, SNA, and ONT
- South Florida: MIA, FLL, and PBI
- Toronto: YYZ and YTZ
- Sao Paulo: GRU, CGH, and VCP
- Rio de Janeiro: GIG and SDU
- Buenos Aires: EZE and AEP
- Melbourne: MEL and AVV

Airport codes are seeded from OurAirports, while metro membership is a curated product decision. OurAirports publishes downloadable airport data as public domain and warns that accuracy is not guaranteed:
<https://ourairports.com/data/>

The database answers only the structural question: are these airports in the same defined metro region? It deliberately does not provide transfer time, disruption, accessibility, or last-service facts. Those volatile facts must come from route adapters.

The same source file also defines AirportReferencePointRegistry version ourairports-reference-points-2026-07-14-v1 for BKK, HND, NRT, and LAX. Each record contains a country code, coordinate, source record ID, source URL, and verification time. These points may center nearby discovery and supply airport-country facts for an entry query; they are not route estimates, transfer durations, or proof of current airport operations. If a required point is absent, discovery or the entry query remains unavailable rather than using an invented coordinate or country.

This separation is a design and safety choice. A durable airport-code mapping is suitable for a bundled database. A time-sensitive claim about HND-to-NRT travel is not.

Architecture and data flows

Application structure

The application follows SwiftUI with MVVM boundaries. Provider protocols isolate commercial and public data sources. Actors own mutable caches. Route selection and risk classification are pure enough to test deterministically.

SwiftUI views | v View models | +--> repositories and policy gates | | +--> proxy providers | +--> demo providers | +--> protected caches | +--> pure route and recommendation engines | +--> versioned result and trace

AirportXRCompanion/App/AppContainer.swift and

AirportXRCompanion/ViewModels/LongHaulExperienceViewModel.swift enforce mode-specific dependencies:

- DEMO and DEBUG stochastic modes use the named itinerary fixtures, minimal demo traveler facts when needed, demo candidates, and TokyoInterAirportDemoTransferProvider.
- LIVE loads only a protected cached itinerary until the user adds or refreshes real flight records. It does not synthesize the reference itinerary or layover-duration candidates.
- OFFLINE loads only protected cached state, marks it stale, and performs no provider refresh.

- LIVE and OFFLINE use UnavailableInterAirportTransferProvider until a current route adapter is configured; they never substitute the Tokyo demo routes.

AirportXRCompanion/Configuration/AppLaunchContext.swift exposes live, demo, offline, and DEBUG stochastic modes. The stochastic DEBUG seed is logged so a test journey can be replayed.

Flight-data flow

Journey or Flights view -> FlightRepository -> configured HTTPS proxy provider -> Cloudflare Worker -> upstream flight provider -> normalized provider metadata -> protected local cache -> freshness-labeled UI -> optional policy-gated learner

Commercial secrets terminate at the Worker. The iOS app receives normalized fields and provenance, not the upstream API key.

Recommendation flow

Itinerary snapshot

- flight provenance
- entry status
- baggage status
- facility evidence
- transfer estimates

-> critical-input gate -> triangular duration distributions -> stable-seed Monte Carlo, $n = 10,000$ -> Wilson 95 percent interval -> SAFE, TIGHT, NOT RECOMMENDED, or REQUIRES CONFIRMATION -> calculation trace

The critical-input gate comes before simulation classification. A numeric result is not used to conceal missing authoritative facts.

Entry-provider and cache flow

Minimal traveler facts + exact itinerary facts -> recursive sensitive-document-field rejection -> Sherpa v3 structured guidance -> compatible-failure Timatic contract adapter -> optional official-domain Gemini link discovery -> normalized evidence kind + provider chain + official links -> exact-query protected cache -> current structured guidance + separate user verification gate -> otherwise REQUIRES CONFIRMATION

Expired structured records and discovery-only links remain inspectable but cannot authorize a landside or city plan. The safety engine never learns a visa rule from previous outcomes.

Delay, offer, and connection-baggage flow

Official flight status

- sourced weather / congestion / rotation / schedule features

-> versioned model validation -> calibrated arrival-delay distribution or UNAVAILABLE -> optional layover scenario input; official status remains unchanged

Current priced airline/NDC offer -> identity + price + source + expiry validation -> HTTPS external handoff -> provider or airline booking; no in-app payment or ticket issuance

Connection-specific airline/order/PNR/offer evidence -> through-check / reclaim-recheck / self-transfer / confirm / unknown -> additive sourced process segments and cutoffs -> HND-to-NRT inter-airport segment when airport codes differ -> existing feasibility engine and calculation trace

The seeded FlightOperationalReferenceFixtures HND-to-NRT object supplies deterministic Book & bags demo/test replay and cannot masquerade as live. No production provider adapter currently supplies these operational-intelligence contracts, so live/offline presentation remains UNKNOWN or unavailable.

Inter-airport flow

Inbound airport differs from onward airport -> classify airport relationship -> look up metro membership -> request current route options -> filter freshness and accessibility -> compute Pareto frontier -> deterministic route ordering -> insert mandatory surface-transfer candidate -> calculate safe onward-airport arrival target -> evaluate optional post-transfer activity

For HND to NRT, Tokyo metro membership opens the inter-airport workflow. It does not itself prove that any surface route is available.

Cross-device reminder flow

Versioned itinerary + current assessment -> only known future leave-by, gate-close, or safe latest-return facts -> stable ReminderIntent identifiers -> explicit versioned destination consent +--> EventKit marked event + exact-time alarm +--> Google Tasks user-OAuth Worker sync + date-only due field

Permission is never requested by a background or routine sync. Unknown or unsafe inputs create no record. A synchronized record can appear on another device only through the user's selected calendar/task account, and continuously revising it while the app is suspended still requires a push/background strategy.

Optional AI explanation flow

Existing sourced trace or facility records -> strict schema + prohibited-domain rejection -> minimized fact IDs and descriptors -> Workers AI ordering only -> all-ID invariant validation -> deterministic renderer of original values and provenance -> original deterministic order on any AI failure

The AI output is presentation metadata. It is never an operational input or a substitute for a missing provider fact.

OCR flow

User selects image -> local Apple Vision text recognition -> advisory parsed fields -> user confirmation

Optional cloud path, only after explicit consent -> crop and re-encode one still image -> enforce size limit -> HTTPS proxy -> normalized text -> no image persistence -> user confirmation

Provider matrix and secret placement

Capability	Current source status	Data mode and cache policy	Secret or entitlement placement	Safe failure behavior
FlightAware AeroAPI	Flight proxy route and iOS adapter implemented	License-dependent flight cache; provenance retained	Worker secret FLIGHTAWAREAEROAPIKEY; never in iOS. AEROAPI_KEY is not accepted by the current Worker.	Typed auth, rate-limit, not-found, server, timeout, and decoding errors; protected cache may remain visible
Amadeus flights or travel services	Worker credential support present	Depends on route-specific provider policy	Worker secrets AMADEUSCLIENTID and AMADEUSCLIENTSECRET	Do not retry authentication loops; retain valid cache
Amadeus hotels	Worker route and ProxyAccommodationProvider source observed; not wired into the current AppContainer screen flow	Memory-oriented current lookup	Worker secrets AMADEUSCLIENTID and AMADEUSCLIENTSECRET	Show unavailable or cached information; never invent availability
Amadeus activities	Worker endpoint source observed; no active iOS adapter or screen wiring observed	No current end-to-end cache claim	Worker secrets AMADEUSCLIENTID and AMADEUSCLIENTSECRET	Keep optional discovery absent rather than blocking the required transfer
Affordability and currency conversion	Airport-local affordability catalog is wired to Layover & Transit; Frankfurt v2 refreshes FX without a key	Venue bands are named demo fixtures; FX quote is memory-only and carries its rate date separately	No secret for Frankfurter; future fare/merchant sources belong behind the Worker and their license policies	Retain labeled local bands; never convert a preview band into a live merchant-price claim; unknown airports remain empty
Entry requirements	Sherpa v3 primary, Timatic normalized backup, official-domain Gemini discovery, iOS proxy/cache, and bundled official-link fallback implemented; no live credential result	Exact-query protected cache; provider-bounded expiry; stale display-only; discovery cannot authorize	Worker secrets SHERPAAPIKEY; TIMATICAPIURL, TIMATICAPIKEY, optional TIMATICSERVICE_TOKEN; optional GEMINIAPIKEY	Stop on auth/rate failure; use compatible fallback only; always require official verification; search/AI never decides entry
Apple MapKit	Native nearby and map capability	Platform-managed; no broad place persistence claim	Apple platform entitlement; no app API key	Map or discovery can be unavailable without changing the safety result
Google Places	Policy registry entry only; no active adapter observed	Memory only; no broad persistence; place identifier exception subject to current policy	GOOGLEPLACESAPI_KEY at Worker if an adapter is added	Do not train or persist restricted place content
WeatherKit	Native current-weather adapter for the active airport	30-minute in-memory freshness	Apple WeatherKit entitlement, not an embedded API key	Unavailable weather remains unavailable; it is not silently substituted
Delay/congestion/rotation intelligence	Provider-neutral feature, model, distribution, provenance, and validation services implemented; no production feature or model adapter	Reject missing/stale/identity-mismatched features and expired predictions; no operational cache claim	Provider-specific configuration at Worker/platform boundary when added	Return UNAVAILABLE and preserve official status

Capability	Current source status	Data mode and cache policy	Secret or entitlement placement	Safe failure behavior
Airline capacity and baggage facts	Evidence-tier normalizer and adjacent-leg Book & bags cards implemented and UI-test verified; no production airline/order adapter	Source-bound fact expiry; missing expiry is unknown; order session is ephemeral and non-Codable	Airline/authorized distribution relationship, not a universal public key	Seat map/fare signals stay proxies; live/offline cards remain unknown without operating-airline fact
Flight offers and booking handoff	Exact-match demo preview screen plus current-offer and HTTPS external-handoff contracts implemented and UI-test verified; no live adapter	Offer/handoff valid only through provider expiry; demo says NOT LIVE OR BOOKABLE; no ticket/payment persistence	Future airline/NDC/aggregator credentials at Worker or approved SDK boundary	Reject stale/invalid/identity-mismatched offers; no in-app payment or issuance
GTFS static	Adapter-ready policy and bundled sample support	Versioned persistent static feed	Per-feed URL or provider configuration; usually no universal key	Retain the last licensed static version with a visible version date
GTFS-Realtime	Planned live route input	Memory only	Per-provider endpoint and key where required	Mark route stale or unavailable; do not claim fastest
HND-to-NRT transfer	Tokyo demo provider is injected only in demo and stochastic modes; live and offline inject UnavailableInterAirportTransferProvider	Demo ranges are not persisted as live route facts	No current live key or route adapter is configured	Keep current inter-airport travel time unresolved and withhold fastest, arrival-target, and interval-visit claims
Transitland	Optional configuration documented; no active adapter observed	Must follow provider terms	TRANSITLANDAPI/KEY at Worker if implemented	Fall back to demo or unavailable with explicit labeling
HND facility registry	Versioned official-link registry implemented	Versioned persistent public facts; availability remains separate	No key	Show source date and request confirmation for availability
Apple Vision OCR	Implemented local advisory path	Local processing; no image persistence	No key	Let the user enter or confirm fields manually
Google Cloud Vision OCR	Optional proxy path, default-off	No image persistence; one bounded request after consent	Worker secret GOOGLEVISIONAPI_KEY	Fall back to local OCR or manual entry
Cloudflare Workers AI explanation	POST /v1/ai/explain implemented and mock-tested; no iOS affordance or live inference result	no-store; no native-model training; minimized fact descriptors only	Non-secret Worker AI binding env.AI; no iOS key	Return original deterministic trace/order when binding, quota, model, or schema fails
Gemini entry-link discovery	Optional official-link fallback implemented; no live call observed	Short provider-bounded cache only when used; prose discarded; no eligibility or training	Worker secret GEMINIAPI/KEY; allowlisted domains are non-secret deployment config	Keep only allowlisted government HTTPS citations; otherwise bundled links or unavailable

Capability	Current source status	Data mode and cache policy	Secret or entitlement placement	Safe failure behavior
Apple Calendar reminders	Planner, EventKit adapter, default-off Settings control, active-app refresh/cleanup, and status UI implemented and covered by the passing build/unit suite	EventKit events plus versioned stable bindings after opt-in	User-granted full Calendar permission; no API key	No consent means no read/write/prompt; demo/stochastic writes blocked; exact derived time only; cross-device account delivery remains external acceptance
Google Tasks reminders	iOS proxy adapter and protected Worker idempotent-sync route implemented; production OAuth/runtime not yet established	User account task plus stable marker; Worker does not persist token; API due value is date-only	Short-lived user OAuth bearer with tasks scope; no API-key fallback	Fail before mutation if consent/OAuth/idempotency scan is incomplete; exact time remains in notes only

Configuration boundary

The iOS project contains only proxy base URLs:

- Config/Secrets.xcconfig is the ignored local configuration location.
- Config/Secrets.xcconfig.example contains placeholders.
- Config/Debug.xcconfig and Config/Release.xcconfig consume the non-secret client configuration.
- AirportXRCompanion/Configuration/AppConfiguration.swift accepts HTTPS proxy URLs; localhost HTTP is allowed only for DEBUG development.

Provider secrets belong in the Cloudflare Worker environment, not in the Xcode project, application bundle, source repository, UserDefaults, logs, or screenshots. Cloudflare documents Worker secret handling here:

<https://developers.cloudflare.com/workers/configuration/secrets/>

The checked-in Worker currently recognizes AMADEUSCLIENTID, AMADEUSCLIENTSECRET, FLIGHTAWAREAEROAPIKEY, SHERPAAPIKEY, TIMATICAPIURL, TIMATICAPIKEY, optional TIMATICSERVICETOKEN, optional GEMINIAPIKEY, and optional GOOGLEVISIONAPIKEY.

GOOGLEPLACESAPIKEY and TRANSITLANDAPIKEY remain planned until matching adapters exist. Do not provision unused secrets. Workers AI uses the non-secret AI binding rather than an application key. Google Tasks uses the person's short-lived OAuth bearer; it must never fall back to an API key. An iOS Google OAuth client ID is public configuration, while refresh state belongs in Google Sign-In/Keychain storage.

Worker authentication and quota boundary

The current Worker exposes /health publicly without credentials and protects every /v1 route before provider or quota work. In production:

- REQUESTAUTHMODE defaults to cloudflare-access.
- The Worker verifies the Cloudflare Access JWT's RS256 signature, issuer, audience, expiry, not-before value, and subject.
- DEPLOYMENTENVIRONMENT = *production* rejects REQUESTAUTH_MODE = none.
- The APIRATELIMITER binding hashes the authenticated subject with a route category.
- wrangler.toml configures 60 requests per 60 seconds as a coarse abuse guardrail.
- A required missing or failed limiter returns 503; a quota rejection returns 429 with Retry-After.

- Provider and surfaced external URLs must use HTTPS and cannot embed credentials.
- Redacted logging excludes provider payloads, credentials, detailed queries, and profile content.

This secure server boundary is not yet an end-to-end live client. The iOS client does not acquire or attach a Cloudflare Access session or assertion, so a correctly protected live deployment returns 401 until that native flow is implemented. Apple App Attest or equivalent device proof is also not implemented. These limitations are recorded in `Documentation/SecurityReview.md` and `Backend/README.md`.

Provider documentation:

- FlightAware AeroAPI: <https://www.flightaware.com/aeroapi/portal/documentation>
- Amadeus hotel resources: <https://developers.amadeus.com/self-service/apis-docs/guides/developer-guides/resources/hotels/>
- Amadeus destination experiences: <https://admin.developers.amadeus.com/self-service/apis-docs/guides/developer-guides/resources/destination-experiences/>
- Frankfurter currency API: <https://frankfurter.dev/> (keyless daily reference exchange rates; the returned rate date is shown)
- Google Places policy: <https://developers.google.com/maps/documentation/places/web-service/policies>
- Apple MapKit: <https://developer.apple.com/documentation/mapkit>
- Apple WeatherKit: <https://developer.apple.com/weatherkit/>
- Google Cloud Vision OCR: <https://docs.cloud.google.com/vision/docs/ocr>
- Apple Vision text recognition: <https://developer.apple.com/documentation/vision/recognizing-text-in-images>

No provider credential value or live commercial response was inspected for this report.

Data modes, caching, and license-aware learning

Visible data modes

`AirportXRCompanion/Models/LongHaul/ProviderModels.swift` and the shared interface components distinguish:

Mode	Meaning	Allowed presentation
LIVE	Current provider response inside its freshness policy	Show source and observed timestamp; may support a claim only if all critical inputs are current
CACHED	Previously fetched data still within the relevant cache policy	Show cached label and timestamp; do not imply a new provider check
STALE	Retained data older than the freshness threshold	Keep context visible but withhold time-sensitive safety or fastest-route claims
DEMO	Bundled or generated research fixture	Show DEMO - NOT LIVE; use for onboarding, tests, and reproducible evaluation
UNAVAILABLE	No acceptable source or cache	Show the missing field and a confirmation or retry action

Freshness is part of the recommendation input, not cosmetic metadata.

For the long-haul feature, DEMO is also a dependency boundary. Only demo or stochastic modes create named reference itinerary, candidate, profile, and HND-to-NRT transfer fixtures. Live and offline keep missing state unavailable; they do not upgrade missing data by borrowing a demo duration.

Cache boundaries

AirportXRCompanion/Aviation/Caching/FlightCache.swift is an actor-backed Application Support cache. Writes are atomic, files use complete-until-first-authentication protection, and cache files are excluded from backup.

AirportXRCompanion/Persistence/ItineraryCache.swift uses:

- Schema version 2.
- A recorded policy version.
- A recorded model version.
- Stable sorted-key encoding.
- Protected atomic persistence.

Flight freshness must come from provider/source expiry metadata and the matching policy, not one universal age. The UI exposes the actual source timestamp and state. A route, regulatory record, priced offer, or airline fact may therefore expire on a different boundary from a flight status.

EntryRequirementCache is a separate protected cache because regulatory guidance cannot safely use a broad flight-age rule. It records only a one-way exact-query fingerprint and normalized assessment, binds the snapshot to the current provider-policy version, writes atomically with complete-until-first-authentication protection, and excludes the file from backup. The provider's bounded expiry controls authority. Expired records retain provenance for display but cannot unlock landside/city feasibility.

Operational delay predictions, airline capacity facts, flight offers, booking handoffs, and connection-baggage facts accept cache use only through their recorded source/provider expiry. Missing expiry is unknown freshness and is rejected. A priced offer is not a reservation, and a cached seat map is not evidence against overbooking.

Provider-policy registry

ProviderPolicyRegistry version provider-policy-2026-07-14-v3 records retention and training permissions by source. Current patterns include:

- Versioned persistent public facility facts.
- Versioned persistent GTFS static data.
- Memory-only GTFS-Realtime data.
- Memory-only WeatherKit data.
- Protected local entry records.
- Separate protected policies for Sherpa, Timatic, and Gemini official-link discovery.
- No-store Workers AI explanation metadata.
- Memory-only Google Places content with no broad persistence.
- License-dependent flight caching.
- User-owned outcome data as a separate learning source.

Current commercial provider policies contain no authorized training purposes. The ability to learn from eligible live data exists, but the shipped catalogs deny every current commercial source until a deployment operator verifies and records a contract for one exact purpose. The mere availability of flight, place, hotel, or entry content therefore does not authorize model training.

Exact-purpose dual authorization for on-device learning

AirportXRCompanion/Models/LongHaul/ProviderModels.swift, AirportXRCompanion/Aviation/Repositories/FlightRepository.swift, and AirportXRCompanion/Services/OnDevicePredictionService.swift implement a default-deny chain. A commercial flight observation reaches the local delay learner only when every condition holds:

1. Personalization is enabled.
2. The repository requests the exact purpose flightDelayOutcome; a broad training flag is insufficient.
3. The bundled ProviderPolicyCatalog contains the source policy ID at the matching policy version and permits flightDelayOutcome.
4. Normalized source metadata independently reports the same policy ID and version, providerTrainingAllowed = true, and the same exact purpose.
5. The source is live and non-demo.
6. The adapter provider identifier and stable provider record identifier are nonempty.
7. Both scheduled departure and actual departure are present, making the delay outcome resolved.
8. The authorization delivered to the learner still matches the source policy ID, policy version, and exact purpose.

The repository groups only authorized records by ProviderTrainingAuthorization before forwarding them. The service deduplicates refreshes using a deterministic one-way StableEntityID derived from the policy version, policy ID, stable provider record ID, and scheduled departure. It persists aggregate delay residual statistics and one-way observation identifiers, not the provider payload.

User-owned walking duration, transit duration, reported delay, and recommendation feedback remain independently eligible under the user-outcomes policy. Entry and visa requirements have no training purpose. Learned output may rank equally safe options, but it cannot weaken the deterministic safety floor or authorize a landside or city visit.

Backend/src/index.js and the bundled iOS registry both use provider-policy-2026-07-14-v3. Every currently configured commercial policy in both catalogs reports no permitted purpose. Live provider data can train after an explicit contract-backed change in both catalogs for the same exact purpose; no such commercial authorization, credential-backed observation, or live learning event is claimed here. Entry rules and AI explanations have no training-purpose enum case and cannot become native policy through personalization.

Advanced algorithm roadmap

The current stable-seed triangular Monte Carlo design favors traceability. More advanced methods are preferred when licensed data, calibration evidence, and failure withdrawal rules exist:

Method	Intended use	Why it is preferable to another magic scalar	Current status
Gradient-boosted quantile regression or survival/competing-risk model	Arrival-delay p10/p50/p90 and exceedance probabilities from weather, congestion, rotation, schedule, route, and carrier context	Represents asymmetric delay and cancellation/diversion outcomes instead of one average minute	Provider-neutral distribution contract implemented; no production model
Hierarchical Bayesian partial pooling	Sparse global route/carrier/airport/hour personalization	Shrinks thin samples toward broader evidence instead of memorizing them	Research recommendation

Method	Intended use	Why it is preferable to another magic scalar	Current status
Rolling conformal calibration	Horizon- and disruption-regime-aware interval correction	Measures empirical coverage and can widen or withdraw a model when drift breaks calibration	Research recommendation
Shared-factor or copula simulation	Correlated inbound delay, airport queue, surface transport, and onward disruption	Avoids optimistic independence when one storm or congestion event affects several segments	Research recommendation; current layover segments are independent
CVaR plus chance constraints	Tail cost of a missed flight, separate-ticket replacement, overnight exposure, accessibility, and checked-bag consequences	Preserves worst-tail consequence beside success probability rather than hiding it in one score	Research recommendation
Time-dependent RAPTOR or connection-scan plus time-dependent A*	Public transit, road, and terminal routing with departure times, last service, outages, and accessibility	Uses the service actually catchable at that time rather than a static fastest-minute figure	Pareto/lexicographic routing exists; live regional adapter remains pending
Pareto frontier with lexicographic tie-breaks	Fastest, accessible, least-walking, fewest-level, simplest, and low-crowd route views	Keeps tradeoffs visible and deterministic instead of summing opaque penalties	Implemented for route-choice boundaries

Evaluation should record pinball loss for quantiles, CRPS for distributions, Brier/reliability curves for exceedance probabilities, conformal empirical coverage, and drift by airport, carrier, route, horizon, and disruption regime. A model without current calibration evidence returns UNAVAILABLE or wider uncertainty. The forecast model produces evidence; the versioned decision policy owns the 0.90/0.70 action boundaries.

API failure behavior

AirportXRCompanion/Aviation/Networking/FlightAPIError.swift defines typed conditions for:

- Not configured.
- Invalid request.
- Authentication failure.
- Rate limiting with optional retry information.
- Not found.
- Offline.
- Server failure.
- Invalid response.
- Response too large.
- Decoding failure.
- General unavailability.

AirportXRCompanion/Aviation/Providers/ProxyFlightDataProvider.swift applies:

- A 15-second request timeout.
- A 1 MB maximum response size.
- JSON content-type validation.

- Explicit mappings for HTTP 400, 401, 403, 404, and 429.
- Retry-After parsing for rate limits.

AirportXRCompanion/Aviation/Repositories/FlightRepository.swift follows these failure rules:

Failure	Repository and UI response	Rationale
Proxy not configured	Flight search may use a deliberately configured, labeled demo provider; long-haul live/offline keeps its itinerary or route unavailable	Missing configuration must not synthesize an operational journey
Offline or timeout	Use cache when available; label CACHED or STALE	The itinerary remains useful while freshness remains honest
401 or 403	Stop provider fallback that would repeat the same authentication problem	Prevent retry storms and credential leakage
404	Show no matching live record; preserve unrelated cached itinerary context	Absence is not a zero-duration or successful result
429	Honor Retry-After; keep eligible cache; stop aggressive fallback	Respect provider limits and avoid amplifying failure
5xx	Preserve cache and allow bounded retry or backoff	A transient server fault must not erase known context
Invalid content type, oversized body, or decoding failure	Reject the response and keep prior valid cache	Malformed data cannot enter the safety calculation
Partial fields	Mark fields as not supplied and request confirmation if critical	Missing values must not be converted to zero

The Worker applies an earlier server-side failure boundary. It authenticates protected requests before quota or provider work; validates flight, airport, geographic, entry, and OCR inputs; bounds upstream time and body size; validates content type and normalized shape; propagates upstream 429 and bounded Retry-After without fallback; and permits FlightAware-to-Amadeus flight-search fallback only for compatible transient failures. Status-by-ID and airport boards remain FlightAware-only because provider identifiers and contracts are not interchangeable.

Backend/test/worker.test.js declares 37 deterministic mocked tests. In addition to Access JWT, fail-closed quota, malformed/oversized upstream, timeout, redaction, provider-policy, and HTTPS boundaries, the source covers Sherpa/Timatic fallback, stale/auth entry behavior, allowlisted Gemini citations, constrained AI explanation, and Google Tasks consent/OAuth/idempotency. It does not directly exercise /v1/scene-ocr or its consent flag. The tests make no live provider, Google account, calendar, or model calls and use no real credential values. The synchronized 37-pass log is recorded below.

For inter-airport planning, a stale route may be displayed for orientation, but it cannot support Fastest now or a SAFE interval-activity recommendation.

Industry design research

Primary-source findings

Apple's profile of Flighty describes an interface that keeps important flight information front and center, draws on airport signage conventions, and recognizes that travelers may lose connectivity. These patterns directly support a high-salience next action, compact flight identity, familiar route structure, and offline-aware state. Source: <https://developer.apple.com/news/?id=970ncww4>

Flighty's own product presentation centers live flight status, airport information, alerts, and a cohesive trip view. It is relevant as a popular independent flight-tracking design reference, not as evidence that every Flighty interaction should be copied. Source: <https://flighty.com/>

Fly Delta describes a Today experience organized around day-of-travel content, boarding pass access, flight status, gate information, airport maps, and offline use. The relevant pattern is progressive focus on what matters now. Source: <https://www.delta.com/us/en/delta-digital/mobile>

Triplt consolidates reservations into one itinerary and exposes airport maps, nearby places, and Navigator guidance. Its Go Now feature provides contextual departure guidance using the trip and current conditions. The relevant patterns are itinerary continuity and a clear leave-by action. Sources: <https://www.tripit.com/web/free> and <https://help.tripit.com/en/support/solutions/articles/103000063349-go-now>

Apple's Human Interface Guidelines inform stable tab navigation, large and clearly labeled controls, accessibility, and glanceable live status:

- Tab bars: <https://developer.apple.com/design/human-interface-guidelines/tab-bars>
- Buttons: <https://developer.apple.com/design/human-interface-guidelines/buttons>
- Accessibility: <https://developer.apple.com/design/human-interface-guidelines/accessibility/>
- Live Activities: <https://developer.apple.com/design/human-interface-guidelines/live-activities>

GTFS Schedule and GTFS-Realtime define the standard separation between durable transit schedule data and time-sensitive updates. That separation supports persistent versioned static feeds and memory-only live updates:

- GTFS Schedule reference: <https://gtfs.org/documentation/schedule/reference/>
- GTFS-Realtime reference: <https://gtfs.org/documentation/realtime/reference/>

The HND facility and transfer research uses airport-operator pages rather than crowd-sourced claims:

- HND to NRT access: <https://tokyo-haneda.com/en/access/narita/index.html>
- Work boxes: https://tokyo-haneda.com/en/service/facilities/work_box.html
- Hotels: <https://tokyo-haneda.com/en/service/facilities/hotel.html>
- Shower rooms: https://tokyo-haneda.com/en/service/facilities/shower_room.html
- Facility index: <https://tokyo-haneda.com/en/service/facilities/index.html>
- Japanese visa-exemption information: https://www.mofa.go.jp/j_info/visit/visa/short/novisa.html

Design rationale matrix

Design choice	Research or implementation basis	Rationale	Failure avoided
Put one safest next action first	Flighty front-and-center information; Fly Delta Today; Triplt Go Now	A stressed traveler should immediately understand whether to transfer, wait, confirm, or leave	A feature grid hiding the time-critical action
Keep Return to Gate available across every root tab	Apple button and accessibility guidance; six-tab simulator audit	A full-width labeled control opens the route from Journey, Flights, AR Guide, Map, Transit, or Settings without depending on the current navigation stack	A traveler hunting through tabs when boarding time changes
Use red as an urgency cue with text and a walking symbol	Apple color guidance; system prominent button styling	The user requested high salience, while redundant text and icon meaning avoid color-only communication; the control does not use the destructive semantic role	Red being mistaken for deletion or becoming inaccessible to color-blind users
Pair the visual terminal map with an expanded step list	Apple accessibility and SwiftUI accessibility-modifier guidance	VoiceOver users receive ordered place, level, maneuver, distance, and route-mode information rather than an unlabeled drawing	AR or map geometry becoming the only way to reach a gate
Keep main-flow copy short and move calculations behind Why this plan?	Travel-day glanceability and progressive disclosure	"We don't have enough information yet" communicates the result; detailed unresolved inputs remain available on demand	Internal model language overwhelming a passenger
Keep one continuous journey timeline	Triplt itinerary consolidation; airline day-of-travel patterns	Flights, surface transfers, airport processes, and optional activities belong to one sequence	Treating HND-to-NRT as an unrelated local trip
Display local airport time zone explicitly	Aviation itinerary conventions and current domain model	Arrival, gate close, opening hours, and transit schedules can otherwise be compared incorrectly	Silent time-zone arithmetic errors
Preserve six stable native root tabs	Apple tab-bar guidance; RootTabView source	Stable destinations provide orientation while Journey stays time-sensitive	Replacing navigation whenever state changes
Use airport-signage hierarchy	Apple's Flighty design profile	Large codes, route direction, gate, and status are recognizable under time pressure	Decorative travel imagery overpowering operational context
Label LIVE, CACHED, STALE, DEMO, and UNAVAILABLE in words	Offline-aware travel products; provenance model	Data authority must be understandable without opening a trace	Demo or stale information appearing live
Isolate demo fixtures by launch mode and dependency injection	AppContainer and LongHaulExperienceViewModel source	Demo and stochastic remain reproducible while live and offline preserve missingness	A polished reference itinerary or transfer being mistaken for a live result
Encode status with word, symbol, and color	Apple accessibility guidance	Meaning survives color-vision differences, grayscale, and high-stress scanning	Color-only safety communication

Design choice	Research or implementation basis	Rationale	Failure avoided
Use a 52-point primary action target	Apple button guidance establishes at least a comfortable touch target; UI review specifies 52 points	The main transfer or confirmation action should be easy to hit while walking or carrying luggage	Small dense dashboard controls
Use semantic system surfaces; reserve teal for interaction	Native iOS visual hierarchy	System colors adapt to appearance and contrast; teal consistently signals an action	Using brand color as an unsafe proxy for safety
Let access layers adapt to Dynamic Type	Apple accessibility guidance	Critical content must remain readable at larger text sizes	Fixed-height cards clipping times or warnings
Show the result before the formula	Progressive disclosure and travel-app glanceability	Most users need the action first; expert users can inspect the trace	A probability worksheet becoming the primary interface
Put mandatory HND-to-NRT transfer before optional places	Itinerary continuity and safety policy	The onward airport is the binding obligation	Recommending sightseeing while the required transfer is unresolved
Withhold fastest wording for demo, stale, or incomplete routes	Provenance policy and GTFS static versus realtime separation	Fastest is a current comparative claim, not a visual flourish	False precision from fixed demo durations
Separate official facility evidence from live availability	Airport-operator research and facility registry	A facility can exist but be closed, full, in another access zone, or ineligible	Treating a directory listing as a reservation
Keep booking external	Current prototype scope and provider boundaries	The app can guide without storing payment or asserting fulfillment	Expanding privacy and liability surface
Put baggage-transfer state on each connection, not only in profile settings	Airline order/offer evidence model and connection builder	Through-check, reclaim/recheck, and separate-ticket handling can differ for every pair of legs and materially change required time	Assuming one baggage choice applies to the entire itinerary
Distinguish order/PNR, offer, seat map, and fare inventory evidence	IATA NDC/Amadeus source research and capacity normalizer	These records answer different questions and cannot all prove a ticket, oversale state, or bin space	Turning shopping or seat-map UI into an airline guarantee
Show an expiring offer with an external airline/provider handoff	Provider-bound offer contracts	Search and comparison can remain useful without collecting payment or claiming ticket issuance	Stale price presented as booked travel
Keep official status visually authoritative over delay prediction	Flight-delay context service	A calibrated scenario can improve connection planning but must not overwrite airline operations	A model prediction appearing to be a gate/airline instruction
Use exact derived times for reminders	EventKit/Google Tasks planner and Triplt-style contextual departure guidance	The same leave-by, gate-close, and latest-return evidence should drive both app and cross-device cues	Hidden fixed reminder offsets diverging from the plan
Explain Google Tasks date-only behavior before opt-in	Google Tasks API contract	Users should understand that the task due field cannot guarantee an exact alert time	Overpromising cross-device notification precision
Let AI order facts but render values deterministically	Workers AI constrained-output architecture	An optional explanation can improve scanning while provenance and decision authority remain in code	Hallucinated operational values or visa conclusions

Design choice	Research or implementation basis	Rationale	Failure avoided
Keep thresholds versioned and contextual, never random	Safety-policy audit/replay requirement	Policy can evolve with calibration and consequence evidence while one snapshot stays reproducible	The same evidence producing different safety labels by chance
Keep OCR local by default and advisory	Apple Vision capability and privacy design	Recognition reduces typing without transferring documents or silently changing the itinerary	Cloud exposure and unconfirmed extraction errors
Use Pareto and lexicographic route ranking	InterAirportTransferPlanner source	Duration, transfers, and walking remain visible and deterministic	An opaque scalar score hiding accessibility tradeoffs
Show a fallback transfer option	Inter-airport failure analysis	A missed train, traffic spike, or service disruption should not collapse the plan	Single-route fragility
Gate optional activity by full round-trip timing and opening hours	Safety engine and candidate factory	A place is only useful if the traveler can complete it and return safely	Recommending a closed or unreachable interval visit

Eight adapted ASCII wireframes

The first seven wireframes adapt Documentation/UIUXReview.md to the current evidence. The eighth records the flight-offer and per-connection baggage screen. They use ASCII only so the canonical report remains reproducible across Markdown and PDF renderers. They are behavioral specifications; the separate acceptance screenshot set records the rendered simulator interface.

Wireframe 1 - Journey overview, BKK-HND-LAX

CODE - TEXT

```
+-----+
| Journey                                [DEMO] |
| Local airport time: Tokyo (JST)         |
+-----+
| BKK -> HND -> LAX                      |
| TG660           HND layover 6h 10m      NH106 |
|                                           |
| NEXT                                           |
| Stay in HND and confirm onward gate information |
| Data: DEMO - NOT LIVE                      |
|                                           |
| [ View safest plan ]                      |
+-----+
| Timeline                                |
| 1 Arrive HND                             |
| 2 Required airport processes              |
| 3 Rest or work candidate                  |
| 4 Security and gate walk                  |
| 5 Gate close                             |
+-----+
| Journey | Flights | AR | Map | Transit | Settings|
+-----+
```

Rationale:

- The route and next action occupy the first scan line because flight-focused products prioritize day-of-travel status.



- Local time is explicit because a multi-time-zone journey otherwise invites deadline mistakes.
- DEMO - NOT LIVE sits next to the actionable content, not in a distant disclaimer.
- The timeline keeps required processes and the optional candidate in one sequence.
- The six destinations remain stable for orientation.

Wireframe 2 - Layover decision

CODE - TEXT

```
+-----+
| Layover plan                                [DEMO] |
| 4h 42m until modeled gate close              |
+-----+
| [SAFE] Quiet rest inside HND                 |
| Wilson 95%: 92% to 94%                      |
|                                               |
| Why this leads                             |
| - Remains in the airport                    |
| - Required return steps are modeled         |
| - Conservative lower bound clears 90%       |
|                                               |
| [ Choose this plan ]                        |
+-----+
| Other candidates                           |
| [TIGHT] Work pod - landside; border return check |
| [NO] City visit - not recommended           |
+-----+
| [ Show calculation trace ]                  |
+-----+
```

Rationale:

- The action label precedes the probability details.
- The interval is shown because a point estimate alone conceals simulation uncertainty.
- SAFE is paired with a word and symbol rather than a color-only chip.
- The alternative list explains why a more attractive activity may be worse.
- A separate trace control supports progressive disclosure.

Wireframe 3 - Airport services, hotel, shower, and work

CODE - TEXT

```
+-----+
| HND services                                [OFFICIAL]|
| Facility facts are not live availability      |
+-----+
| Work box - Terminal 3                      |
| Access: LANDSIDE / GENERAL AREA            |
| Published hours: 07:00-21:30                |
| [ Confirm border return ] [ Open official page ] |
+-----+
| Transit hotel - Terminal 3                  |
| Access: AIRSIDE                             |
| Eligibility and room: CONFIRM                |
| [ Open official page ]                      |
+-----+
| Shower - Terminal 3 arrival lobby           |
| Access: LANDSIDE                             |
| Published: 24 hours; reservations not accepted |
| Current wait: UNAVAILABLE                    |
| [ Open official page ]                      |
+-----+
```

Rationale:

- The work-box access label is corrected to landside or general area from the stale UI review description.
- Access zone is placed beside each facility because border and security re-entry time can dominate the activity.
- Official existence, published hours, eligibility, room inventory, and current wait are separate fields.
- External links preserve booking and current confirmation with the authoritative operator.

Wireframe 4 - Entry, baggage, and border confirmation

CODE - TEXT

```
+-----+
| Confirm required steps                      [?] |
| We cannot calculate a safe plan yet         |
+-----+
| Entry permission                            |
| Status: REQUIRES CONFIRMATION                |
| Official reference: Japan MOFA              |
| [ Review official guidance ]                 |
+-----+
| Checked baggage                             |
| Is it through-checked to the onward flight? |
| [ Yes ] [ No ] [ I am not sure ]             |
+-----+
| Onward airport and terminal                 |
| NRT terminal: UNAVAILABLE                    |
| [ Enter or refresh ]                         |
+-----+
| [ Continue after confirmation ]              |
+-----+
```

Rationale:

- The screen explains why a safety result is withheld rather than showing a disabled score.
- The app asks about baggage in plain language and supports uncertainty.
- The official entry link informs but does not claim an eligibility decision.
- Unknown terminal data remains unknown; it is not silently assigned a zero transfer time.

Wireframe 5 - Calculation trace

CODE - TEXT

```
+-----+
| Why this result                                |
| Policy: layover-safety-2026-07-14-v1         |
| Data: DEMO - NOT LIVE                        |
+-----+
| Result: [TIGHT]                              |
| Estimated success: 86%                      |
| Wilson 95% interval: 85% to 87%            |
| Trials: 10,000                              |
+-----+
| Modeled segments                             |
| Deplane          min / likely / max         |
| Border           min / likely / max         |
| HND -> NRT       min / likely / max         |
| NRT check-in     min / likely / max         |
| Security + gate   min / likely / max         |
| Activity         min / likely / max         |
+-----+
| Seed inputs                                         |
| Itinerary ID + input revision + snapshot version |
| + policy version                                  |
| [ Copy evidence summary ]                        |
+-----+
```

Rationale:

- Result, interval, and trial count come before implementation details.
- Versioned policy and data authority make the calculation auditable.
- Segment ranges expose the assumptions that matter most to the traveler.
- The trace identifies seed inputs without exposing a secret or personal document.

The percentages shown here are illustrative wireframe content, not an observed scenario output.

Wireframe 6 - In-flight advisory OCR

CODE - TEXT

```
+-----+
| Read itinerary image                |
| Local recognition by default        |
+-----+
| [ Select photo ]                   |
|                                     |
| Recognized fields                   |
| Flight: NH6                        |
| Airport: NRT                       |
| Time: 17:00                        |
|                                     |
| Check every field before using it   |
| [ Edit ] [ Confirm fields ]        |
+-----+
| Optional cloud recognition          |
| OFF - sends one cropped still only with consent |
| [ Learn what is sent ]             |
+-----+
```

Rationale:

- Local recognition is the default, reducing document exposure.
- Recognized fields stay editable and unconfirmed.
- The optional cloud path names the bounded payload before consent.
- OCR is a data-entry aid and never an authoritative flight source.

Wireframe 7 - HND-to-NRT transfer and optional interval

CODE - TEXT

```
+-----+
| Transfer HND -> NRT                 [DEMO] |
| Required before onward NH6          |
+-----+
| NEXT                                |
| Confirm border, baggage, and NRT terminal |
| Arrival target at NRT: UNKNOWN       |
| [ Confirm required details ]         |
+-----+
| Leading demo option - not live       |
| Road      likely 92m   range 60-150m   walk 90m |
| [ Accessibility unknown ]           |
|                                     |
| Alternatives                         |
| Rail      likely 96m   range 82-125m   walk 420m |
| Bus       likely 100m  range 70-145m   walk 180m |
| [ Refresh current routes ]          |
+-----+
| Interval after reaching NRT          |
| Places: DISCOVERY ONLY               |
| No visit recommended until transfer, check-in, |
| security, opening hours, and return margin fit. |
| [ View NRT-area places ]            |
+-----+
```

Rationale:

- The airport pair and mandatory nature of the segment are unambiguous.
- Leading demo option - not live avoids an unsupported fastest claim.
- Duration range, walking, transfers, and accessibility remain visible instead of collapsing into one score.
- The NRT arrival target is withheld until critical inputs resolve.
- Optional places appear only after the transfer section and are labeled discovery-only until the full interval is safe.

Wireframe 8 - Flight offer and per-connection baggage

CODE - TEXT

```
+-----+
| Flights and booking support      [DEMO/TESTED] |
| Offer expires 14:05 JST      Source: airline/NDC |
+-----+
| BKK -> HND                      |
| Baggage: checked to HND        |
| Evidence: current order / PNR  |
+-----+
| HND -> NRT connection           |
| [RECLAIM + IMMIGRATION + CUSTOMS + RECHECK]    |
| Bag drop cutoff: SOURCE REQUIRED |
| Required time                   |
| baggage wait + border + customs + landside     |
| + HND-NRT + bag drop + security + terminal     |
| [ Show connection calculation trace ]           |
+-----+
| NRT -> LAX                      |
| Overbooking: UNKNOWN           |
| Overhead-bin space: UNKNOWN    |
| Seat map is not a capacity guarantee            |
+-----+
| Price: provider-confirmed until expiry          |
| [ Continue with airline/provider ] EXTERNAL    |
| No in-app payment or ticket issuance           |
+-----+
```

Rationale:

- The baggage label belongs to a specific connection because through-check can differ between adjacent legs.
- Required reclaim, border, customs, landside, recheck, security, and inter-airport segments are visible instead of being hidden in one connection number.
- The bag-drop cutoff remains a required sourced fact; the card cannot present a positive result when it is missing.
- Overbooking and bin space remain UNKNOWN unless the operating airline supplies an explicit current fact; seat-map availability is labeled as a proxy.
- Offer expiry and source appear next to price, and the primary action clearly leaves the app for airline/provider fulfillment.
- DEMO/TESTED distinguishes the verified seeded screen behavior from a live booking integration.

Figma design status

A Figma foundations file exists and is editable:

- File: <https://www.figma.com/design/MgNQlseRuiBCZvDjXnsvHi>
- Foundations root node 3:31: <https://www.figma.com/design/MgNQlseRuiBCZvDjXnsvHi?node-id=3-31>

Observed foundations include 21 semantic color, spacing, and radius variables; seven SF Pro text styles; status examples; an accessibility contract; and four design-principle cards.

The eight product frames in this report were not created or visually validated in Figma. Figma creation was blocked by the Starter-plan MCP tool-call quota, including a Not permitted to upsert from library error. The foundation file must not be described as a completed product-wireframe file.

User and developer guides

Documentation/UserDemoGuide.md is the passenger-facing demonstration script. It covers the BKK-HND-LAX journey, HND-to-NRT airport change, test-only four-leg/date-line case, itinerary editing, terminal map and AR caveats, HND facilities and nearby discovery, entry verification, OCR, offline cache, stochastic/replay launch, personalization and erase controls, reminder/AI foundations, and acceptance commands. Its status legend distinguishes UI demo, cached/offline, fixture/test only, architecture only, backend contract only, and unavailable behavior so an internet-connected simulator is not mistaken for a live-provider proof.

Documentation/DeveloperGuide.md records the workspace boundary, architecture, policy rationale, dynamic versus deterministic values, advanced-algorithm roadmap, provider/cache/learning contracts, entry fallback, secrets, Worker routes, setup/build/test/launch commands, and extension checklists for providers and metro areas.

Documentation/FlightDelayCapacityAndBookingArchitecture.md is the canonical technical supplement for the newer delay, capacity, offer, and per-connection baggage contracts.

The guides are synchronized with the final local acceptance artifacts. They distinguish verified simulator behavior from live-provider, account-delivery, and physical-device surfaces.

Privacy and security

Current privacy posture

AirportXRCompanion/Resources/PrivacyInfo.xcprivacy declares no collected data and no tracking. It declares the CA92.1 required-reason API use for UserDefaults. The observed prototype has no account requirement and no analytics integration.

The privacy model is:

- Process OCR locally by default.
- Require explicit consent before optional cloud OCR.
- Send at most one cropped and re-encoded still image, bounded to 512 KB, on that optional path.
- Do not stream or persist the cloud OCR image.
- Do not request or store passport numbers or passport scans for entry checks.
- Cache normalized entry guidance only under a one-way exact-query fingerprint; never persist document numbers, MRZ, or scans.
- Store itinerary and cache data with platform file protection.
- Exclude protected caches from backup.

- Provide controls to clear local data and learned outcomes.
- Keep commercial keys at the Worker boundary.
- Keep booking and payment external; keep order/PNR session handles ephemeral and non-Codable.
- Keep Google OAuth access/refresh state in Google Sign-In/Keychain boundaries, never in request bodies, UserDefaults, analytics, or Worker persistence.
- Send Workers AI only minimized fact identifiers/descriptors; reject traveler/entry domains and return no raw model prose.

Threat and mitigation summary

Threat	Relevant asset	Current or required mitigation	Residual limitation
Embedded provider key extraction	Commercial credentials and quota	Store only Worker secrets; iOS uses HTTPS proxy base URLs	Worker operations still require secret rotation and monitoring
Demo data mistaken for current status	Traveler decision	Persistent DEMO - NOT LIVE label and provenance in the trace	Visual regression testing remains necessary
Stale route called fastest	Connection safety	Freshness gate and neutral leading-option wording	Live route adapter is not yet validated
Missing field converted to zero	Recommendation integrity	Typed optional fields and critical-input preemption	Every new adapter must preserve missingness
OCR misread changes itinerary	Flight identity and deadline	Advisory output plus explicit field confirmation	Users can still confirm an incorrect value
Sensitive travel data leaked through logs	Itinerary privacy	Worker redaction excludes credentials, provider payloads, detailed queries, and profiles; local logs keep only bounded diagnostics	Deployment log-sink configuration still needs operator review
Cached data accessible after device loss	Itinerary privacy	Complete-until-first-authentication file protection and device security	Data can be available after first unlock until restart
License-restricted content used for training	Provider rights and model integrity	Exact-purpose agreement between bundled policy and normalized source metadata, plus live/non-demo/resolved/stable-ID gates	Provider terms can change and require registry review
Search or AI treated as visa authority	Entry and traveler safety	Structured/discovery evidence kinds, official-domain allowlist, discarded model prose, current-expiry and user-confirmation gates	Official rules can change after a cached/discovered link was observed
AI invents an operational value	Recommendation integrity	AI receives minimized IDs/descriptors; deterministic code renders every original fact; malformed output falls back	Model/provider availability still needs budget and outage handling
Reminder created from an unsafe or unknown time	Cross-device alerts	Planner accepts only future sourced/derived values and a SAFE latest-return trace with no unresolved inputs	Account sync and suspended-app delivery can still be delayed
Calendar/task synchronization overreaches	Calendar and Google account data	Explicit versioned consent, stable app marker, EventKit marker-scoped reads/deletes, short-lived OAuth bearer, bounded pre-mutation task scan	Full Calendar access is required for EventKit idempotent read/update/delete
PNR, payment, or ticket data persisted by the prototype	Airline account and financial data	Ephemeral non-Codable order session; external HTTPS handoff; no payment or ticket protocol	Future booking expansion requires a new privacy and threat review
Seat-map or fare inventory treated as an airline guarantee	Connection and baggage planning	Evidence-tier normalizer preserves proxy status; operating-airline fact required for oversale/bin guarantee	Authorized data availability differs by airline and contract

Threat	Relevant asset	Current or required mitigation	Residual limitation
Activity recommendation displaces transfer	Onward connection	Mandatory transfer candidate precedes optional activities	Real-world disruption can exceed modeled maxima
Facility listing treated as guaranteed access	Time and access	Separate source, zone, eligibility, and availability	Operator pages can change after registry publication

Verified Worker controls and remaining production boundaries

The current source and deterministic tests verify:

- Cloudflare Access JWT verification for protected routes.
- Production refusal of unauthenticated development mode.
- Fail-closed rate limiting when the required binding is absent or fails.
- HTTPS-only upstream and surfaced external URLs.
- Rejection of sensitive entry document fields.
- Provider-bounded Sherpa/Timatic entry fallback and official-domain-only discovery evidence.
- AI all-facts-preserved rendering, prohibited entry domains, and no raw model-output exposure.
- Google Tasks explicit consent, user OAuth, stable-marker idempotency, and fail-before-mutation page bounds.
- Bounded timeouts, response sizes, content types, and error bodies.
- Default-deny provider training metadata.

A production release would still require:

- A native iOS Cloudflare Access sign-in or session flow; the current protected Worker and current client do not yet interoperate.
- Apple App Attest or an equivalent deployment-approved device proof, if required by the threat model.
- Provider-specific spend controls, durable circuit breakers, and operational alerting beyond the coarse 60-request/60-second abuse guardrail.
- Secret rotation and separation between development and production.
- Transport and response-schema monitoring.
- Provider-term and retention-policy review.
- A privacy review for every new data field.
- A threat-model refresh before production airline/order accounts, expanded booking, payment, or document storage enter scope.

Validation and test evidence

Observed results

Evidence	Observed result	Observation time	Reproducibility anchor
Clean terminal application build	BUILD SUCCEEDED with code signing disabled for the iPhone 17 Pro simulator destination.	2026-07-14T15:48:58-0400	build/final-clean-build.log; SHA-256 15dbfa5fd5f902b3180635ff0796b04735e7b49f57af7880aed6449180706236
Complete iOS unit suite	78 tests passed, 0 failures; TEST SUCCEEDED. This includes itinerary, entry/cache, learning, routing, reminders, operational intelligence, baggage, offers, and safety boundaries.	2026-07-14T15:50:00-0400	build/final-clean-unit-tests.log; SHA-256 66d2ffccf46509391dbc41e8481c5dab03b6eab02f980c6717ac7f37488fe3f
Complete iOS UI suite	13 tests passed, 0 failures; TEST SUCCEEDED: 11 primary UI cases plus two Book & bags cases.	2026-07-14T16:40:07-0400	build/final-full-ui-tests.log; SHA-256 27aa14bc723fb60b5df39b2f16fb1bdf43e93bef2fde603d12fc1f92766299f1; xcresult build/DerivedData/Logs/Test-AirportXRCompanion-2026.07.14_16-35-44--0400.xcresult
Worker static check	Passed after npm ci restored the locked dependency tree.	2026-07-14T16:41:02-0400	build/final-backend-check.log; SHA-256 d5ee07f60c3b8ddda131790ccb410482dced7108f19ca77ecb5b9c24675aac8
Complete deterministic Worker suite	37 tests passed, 0 failures, skips, cancellations, or todos using mocked upstreams and no private payloads.	2026-07-14T16:41:03-0400	build/final-backend-tests.log; SHA-256 749c8eddd8ce650978e6a3cd359bc4c93d52d2375d90afccbce59f1328f73330
Wrangler deployment dry run	Passed; Worker bundled to 138.46 KiB, 33.61 KiB gzip, and exited without deployment. Placeholder Access configuration remains intentionally non-production.	2026-07-14T16:41:04-0400	build/final-wrangler-dry-run.log; SHA-256 cbacc6a7bdaa4cbb1091b3c8dbef141aff11c94108c984d41d2732262b3b4d1d
Ordinary simulator walkthrough	App launched without --uitesting in demo, offline, stochastic, live-requested, and demo inter-airport modes. Five screenshots were visually reviewed; stochastic mode recorded replay seed 17774091554168773787. Live-requested correctly remained fixture-backed without proxy configuration.	2026-07-14T16:42:06-0400	build/final-normal-simulator.log; SHA-256 99d11e915e997a00d8fe493148a568157c714e0159f76bfc3edc11d131f22ff6; Documentation/Screenshots/2026-07-14-final-acceptance/
Rich maneuver and Apple Core Location QA source checks	All application sources type-checked successfully against the iOS 18 Simulator SDK. All application/test Swift sources parsed; Python compiled; shell syntax and git diff --check passed. Core Location driver dry-run emitted xcrun simctl location booted set 35.54975885, 139.78589806.	2026-07-16	Direct terminal observation; no new Xcode build or XCTest result claimed

AirportXRCompanionTests/LongHaulExpansionTests.swift covers the exact Wilson classification boundaries, including lower bound equality at 0.90 and upper bound equality at 0.70. It also exercises the named BKK-HND-LAX fixture, HND-to-NRT planning, mandatory transfer ordering, demo canClaimFastest = false, versioned and sourced airport reference points, and the deny-by-default shipped provider catalog.

The separate 15-test result covers the synthetic exact-purpose allow path, policy/source mismatch denial, deterministic refresh deduplication, disable and erase controls, normalized policy metadata, cache, fallback, rate limiting, freshness, GTFS, notifications, and user-owned personalization. Its allow policy is a deterministic test fixture, not a commercial permission.

The complete UI artifact supersedes the earlier targeted accessibility failure: the revised safe-area clearance and stable accessibility locators were compiled and exercised by the full 13-test run. The screenshots show readable state badges and the HND-to-NRT first-action hierarchy in ordinary launches.

The 37-test Worker transcript is deterministic and mocked. It does not prove a live Cloudflare Access session, provider credential, upstream contract, Gemini citation result, Workers AI inference, or Google account mutation.

The 2026-07-16 maneuver/Core Location revision was not included in the older synchronized build and test artifacts above. At that revision, the data volume had 1.8 GiB free, below the project's 5 GiB build preflight. Source parsing, iOS-SDK type checking, script validation, and dry-run command verification passed, but simulator delivery, new unit cases, UI behavior, and RealityKit arrow appearance still require a fresh build/test/launch after disk space is restored.

Every result proves only its recorded source and command boundary. The local acceptance snapshot establishes a reproducible credential-free application and backend build; it does not convert demo or unavailable provider states into live evidence.

Toolchain recorded for reproduction

- Xcode 26.6, build 17F113.
- iPhoneSimulator 26.5 SDK.
- Build destination: iPhone 17 Pro simulator.
- Target: iOS 18 simulator.
- Swift toolchain: 6.3.3.
- Project Swift language version: 5.
- Node.js: v25.8.1.
- Python: 3.9.6.
- Git commit: not recorded for this snapshot.

External acceptance still requiring credentials or hardware

The synchronized terminal, unit, UI, Worker, dry-run, and ordinary simulator evidence is complete. The following checks cannot be established by the credential-free local snapshot:

- Credential-gated live provider contract tests, including authenticated flight, accommodation, entry, weather, transit, and optional AI responses.
- A native iOS Cloudflare Access session against a protected deployed Worker.
- Apple Calendar synchronization and alert delivery across two signed-in physical devices after explicit user consent.
- Google Tasks delivery after a supported OAuth client is configured; the feature remains visibly disabled without it.
- Physical-device AR tracking, camera permissions, and real terminal localization.

- Live airline-authorized overbooking, baggage-transfer, bag-drop, and capacity evidence.

These are deployment or hardware acceptance surfaces, not failures of the completed credential-free local test suite.

Limitations and open risks

Research and model limits

- Triangular distributions are explainable but may not match empirical tails.
- Segment independence can underestimate correlated disruption, such as an inbound delay causing both immigration congestion and a missed ground departure.
- A maximum in a research distribution is not a physical upper bound.
- Wilson intervals express Monte Carlo sampling uncertainty, not all real-world model uncertainty.
- The 0.90/0.70 thresholds are provisional research policy, not an API-derived fact, certified aviation standard, or empirically validated optimum.
- A 10,000-trial run is deterministic for the same snapshot but remains sensitive to the quality of each input range.
- Entry, baggage, terminal, and facility eligibility can change by traveler and itinerary.

Inter-airport limits

- The HND-to-NRT provider currently supplies demo ranges only.
- The official Haneda page is static published context, not a live disruption-aware router.
- No current GTFS-Realtime or licensed Tokyo journey planner is validated end to end.
- Current source can rank demo options, but it cannot truthfully claim the fastest live route.
- Road, rail, and bus accessibility details are incomplete in the fixture.
- Last service, missed departure wait, terminal-specific boarding point, and disruption behavior need live integration.
- Optional place discovery does not prove opening, admission, luggage acceptance, or timely return.
- Cross-region surface connections need stronger validation than same-metro classification.

Product and operational limits

- The eight Figma product screens are blocked by Starter quota and are not visually validated.
- The complete 13-test UI suite passes in the synchronized acceptance artifact; future source changes require a full rerun.
- The ordinary simulator walkthrough covers five principal launch states; physical-device and credential-backed behavior remains separate.
- No live provider credential or response was inspected.
- The Worker has deterministic authentication, quota, validation, fallback, and policy tests, but no credential-gated live contract result.
- The deterministic Worker suite does not directly test the optional /v1/scene-ocr consent boundary.
- The current iOS client cannot yet acquire or attach the Cloudflare Access session required by the production Worker.
- App Attest or equivalent device proof and provider-specific spend controls are not implemented.

- The hotels adapter is not wired into the current screen flow.
- The activities endpoint lacks an observed active iOS adapter.
- Google Places and Transitland are configuration or policy entries, not active validated integrations.
- WeatherKit is not observed as a wired end-to-end feed.
- Delay/capacity/offer/baggage provider-neutral contracts, seeded demo fixtures, and the Book & bags screen are build- and UI-test-verified, but no production feature/model, airline order, NDC, capacity, or booking adapter is configured.
- Flight offers support an external-handoff contract only. There is no in-app payment, ticket issuance, reservation guarantee, or operator dispatch function.
- No universal public API or current adapter can guarantee overbooking status or overhead-bin space; seat maps and fare inventory remain proxies.
- Structured entry guidance and official links do not guarantee admission; Gemini discovery cannot derive eligibility.
- Workers AI explanation is backend-only, has no iOS affordance, and has no recorded live inference.
- Apple Calendar runtime source passes the local build/unit suite; Google Tasks remains disabled until real OAuth is configured. Cross-device delivery, suspended-app refresh, and exact Google task alerts are not proven.
- The user and developer guides are synchronized to the final local acceptance snapshot.
- There is no recorded Git commit for the evaluated snapshot.

Figma and implementation drift

Documentation/UIUXReview.md supplied the seven conceptual views, but one access-zone detail was stale: the HND Terminal 3 work pod is landside or in the general area. This report adapts that wireframe to the official facility source and current registry. Future design artifacts should be generated from the current source-of-truth report and then visually checked against the implementation.

Reproducibility procedure

Use the following sequence from the repository root, /Users/anony/Downloads/AirportXRCompanion.

1. Preserve the historical unit evidence, then create a synchronized result

```
shasum -a 256 build/current-unit-tests-final.log
```

Expected hash:

```
41013b4d95efa940b1525e02ed9d69e10980c4aa9e1a0eefb80182f278f8de88 build/current-unit-tests-final.log
```

Confirm the retained log ends with TEST SUCCEEDED and 47 tests, 0 failures, then treat it as historical only. Regenerate the project and run the complete current unit target:

```
./Scripts/generate.sh xcodebuild test \ -project AirportXRCompanion.xcodeproj \ -scheme AirportXRCompanion \
-destination 'platform=iOS Simulator,name=iPhone 17 Pro' \ -derivedDataPath build/DerivedData \
CODESIGNINGALLOWED=NO \ -only-testing:AirportXRCompanionTests
```

Record the new total and hash; do not copy the historical 47-test count into final acceptance.

2. Inspect the safety policy and exact boundary tests

```
sed -n '1,260p' AirportXRCompanion/Services/LongHaul/SafetyPolicy.swift sed -n '1,320p'
AirportXRCompanion/Services/LongHaul/MonteCarloLayoverRecommendationEngine.swift sed -n '180,260p'
AirportXRCompanionTests/LongHaulExpansionTests.swift
```

Confirm:

- Policy version layover-safety-2026-07-14-v1.
- $z = 1.959963984540054$.
- Raw required trials round to 9,604.
- Production trials are 10,000.
- SAFE uses lower ≥ 0.90 .
- NOT RECOMMENDED uses upper < 0.70 .
- Equality at upper $= 0.70$ remains TIGHT.

3. Inspect the two reference scenarios

```
rg -n 'demo-bkk-hnd-lax|HND|NRT|TokyoInterAirportDemoTransferProvider' \
AirportXRCompanion/Services/LongHaul/LongHaulReferenceScenario.swift sed -n '30,100p'
AirportXRCompanion/App/AppContainer.swift sed -n '168,230p'
AirportXRCompanion/ViewModels/LongHaulExperienceViewModel.swift
```

Confirm that all fixture values carry demo provenance, the HND-to-NRT transfer precedes any optional activity, and live/offline modes do not create the reference itinerary, candidate durations, minimal demo profile, or Tokyo demo transfer provider.

4. Inspect metro classification and route ranking

```
sed -n '1,260p' AirportXRCompanion/Models/LongHaul/MetroAirportDatabase.swift sed -n '1,360p'
AirportXRCompanion/Models/LongHaul/InterAirportTransferModels.swift
```

Confirm that the metro database supplies membership only, AirportReferencePointRegistry supplies sourced country/coordinate facts rather than travel time, the planner exposes duration, transfers, and walking, and canClaimFastest is false for demo routes.

5. Inspect cache, learning, and privacy gates

```
sed -n '1,260p' AirportXRCompanion/Models/LongHaul/ProviderModels.swift sed -n '1,280p'
AirportXRCompanion/Services/OnDevicePredictionService.swift sed -n '1,260p'
AirportXRCompanion/Aviation/Repositories/FlightRepository.swift sed -n '1,240p'
AirportXRCompanion/Resources/PrivacyInfo.xcprivacy
```

Confirm that current commercial policies do not authorize training and that protected cache behavior remains intact.

Also inspect the exact entry and operational-intelligence boundaries:

```
sed -n '1,280p' AirportXRCompanion/Persistence/EntryRequirementCache.swift sed -n '1,340p'
AirportXRCompanion/Models/FlightOperationalIntelligenceModels.swift sed -n '1,940p'
AirportXRCompanion/Services/FlightOperationalIntelligenceService.swift sed -n '1,260p'
AirportXRCompanion/Reminders/CrossDeviceReminderPlanner.swift
```

Confirm provider-policy-v3 exact-query entry cache, source/expiry rejection, no inferred through-check, external booking only, unknown capacity defaults, and reminder creation only from current derived times.

6. Re-run the exact-purpose learning evidence

```
xcodebuild test \ -project AirportXRCompanion.xcodeproj \ -scheme AirportXRCompanion \ -destination 'platform=iOS Simulator,name=iPhone 17 Pro' \ -derivedDataPath /tmp/AirportXRLearningDerivedData \
CODES/GN/NGALLOWED=NO \ -only-testing:AirportXRCompanionTests/TransitNotificationPredictionTests \
-only-testing:AirportXRCompanionTests/FlightDataTests
```

Expected current count: 15 tests, 15 passed, 0 failed. Confirm that the synthetic authorized policy learns one resolved delay outcome, duplicate refreshes remain one observation, mismatched exact purposes are denied, and the shipped commercial catalogs remain deny-all.

7. Re-run Worker verification

```
cd Backend npm run check npm test npx --yes wrangler@4.36.0 deploy --dry-run \ --outdir /tmp/airportxr-worker-dry-run
```

Backend/test/worker.test.js declares 37 tests at this reconciliation cutoff. Assert the count printed by the fresh run rather than hard-coding it into release automation. These commands use mocked upstreams and no real credentials. Add a direct /v1/scene-ocr consent and payload-boundary test before treating optional cloud OCR as backend-covered.

8. Refresh acceptance artifacts after future source changes

After any application or backend source change, rerun the complete build, 78-test unit suite, current 13-test UI suite, Worker check/tests/dry-run, and ordinary simulator walkthrough from one recorded source snapshot. Capture:

- Full commands.
- UTC timestamps.
- Exit statuses.
- Test totals.
- Tool versions.
- Source or commit hash.
- Log hashes.
- Simulator device and OS.
- Screenshots for each evidence state.

Do not reuse this report's passing snapshot to cover later untested source changes.

Primary references

Platform and product design

- Apple, Behind the Design: Flighty: <https://developer.apple.com/news/?id=970ncww4>
- Apple Human Interface Guidelines, Tab bars: <https://developer.apple.com/design/human-interface-guidelines/tab-bars>
- Apple Human Interface Guidelines, Buttons: <https://developer.apple.com/design/human-interface-guidelines/buttons>
- Apple Human Interface Guidelines, Accessibility: <https://developer.apple.com/design/human-interface-guidelines/accessibility/>
- Apple Human Interface Guidelines, Live Activities: <https://developer.apple.com/design/human-interface-guidelines/live-activities>
- Flighty: <https://flighty.com/>

- Fly Delta mobile app: <https://www.delta.com/us/en/delta-digital/mobile>
- Triplt: <https://www.tripit.com/web/free>
- Triplt Go Now: <https://help.tripit.com/en/support/solutions/articles/103000063349-go-now>

Airport, entry, and transit

- Haneda Airport, Access to Narita Airport: <https://tokyo-haneda.com/en/access/narita/index.html>
- Haneda Airport, Work boxes: https://tokyo-haneda.com/en/service/facilities/work_box.html
- Haneda Airport, Hotels: <https://tokyo-haneda.com/en/service/facilities/hotel.html>
- Haneda Airport, Shower rooms: https://tokyo-haneda.com/en/service/facilities/shower_room.html
- Haneda Airport, Facilities: <https://tokyo-haneda.com/en/service/facilities/index.html>
- Japan Ministry of Foreign Affairs, Visa exemption: https://www.mofa.go.jp/j_info/visit/visa/short/novisa.html
- Sherpa Requirements API: <https://docs.joinsherpa.io/requirements-api/index.html>
- IATA Timatic AutoCheck: <https://www.iata.org/en/services/compliance/timatic/autocheck/>
- IATA Travel Centre: <https://www.iata.org/en/services/compliance/timatic/travel-documentation/>
- Gemini Google Search grounding: <https://ai.google.dev/gemini-api/docs/google-search>
- GTFS Schedule reference: <https://gtfs.org/documentation/schedule/reference/>
- GTFS-Realtime reference: <https://gtfs.org/documentation/realtime/reference/>
- OurAirports data: <https://ourairports.com/data/>

Provider and infrastructure

- Cloudflare Workers secrets: <https://developers.cloudflare.com/workers/configuration/secrets/>
- Cloudflare Workers AI pricing: <https://developers.cloudflare.com/workers-ai/platform/pricing/>
- Cloudflare Workers AI bindings: <https://developers.cloudflare.com/workers-ai/configuration/bindings/>
- Cloudflare Workers AI data usage: <https://developers.cloudflare.com/workers-ai/platform/data-usage/>
- Apple Foundation Models: <https://developer.apple.com/documentation/foundationmodels/>
- Hugging Face Inference Providers pricing: <https://huggingface.co/docs/inference-providers/en/pricing>
- FlightAware AeroAPI documentation: <https://www.flightaware.com/aeroapi/portal/documentation>
- Amadeus flight APIs:
<https://developers.amadeus.com/self-service/apis-docs/guides/developer-guides/resources/flights/>
- Amadeus test data: <https://developers.amadeus.com/self-service/apis-docs/guides/developer-guides/test-data/>
- Amadeus API FAQ and seat-map limitations:
<https://developers.amadeus.com/self-service/apis-docs/guides/developer-guides/faq/>
- IATA NDC: <https://www.iata.org/en/programs/airline-distribution/retailing/ndc/>
- AviationWeather.gov Data API: <https://aviationweather.gov/data/api/>
- FAA NAS Status: <https://nasstatus.faa.gov/>
- FAA TFMData: <https://cdm.fly.faa.gov/faq>
- Amadeus hotels: <https://developers.amadeus.com/self-service/apis-docs/guides/developer-guides/resources/hotels/>

- Amadeus destination experiences: <https://admin.developers.amadeus.com/self-service/apis-docs/guides/developer-guides/resources/destination-experiences/>
- Google Places policy: <https://developers.google.com/maps/documentation/places/web-service/policies>
- Apple MapKit: <https://developer.apple.com/documentation/mapkit>
- Apple WeatherKit: <https://developer.apple.com/weatherkit/>
- Apple EventKit access: <https://developer.apple.com/documentation/eventkit/accessing-the-event-store>
- Google OAuth for native apps: <https://developers.google.com/identity/protocols/oauth2/native-app>
- Google Tasks API: <https://developers.google.com/workspace/tasks/reference/rest>
- Google Tasks task resource and date-only due behavior:
<https://developers.google.com/workspace/tasks/reference/rest/v1/tasks>
- Google Cloud Vision OCR: <https://docs.cloud.google.com/vision/docs/ocr>
- Apple Vision text recognition: <https://developer.apple.com/documentation/vision/recognizing-text-in-images>

Conclusion

The current prototype has a coherent safety and information architecture for long-haul and inter-airport decision support. Its strongest property is that uncertainty is part of the product contract: demo data stays labeled, missing critical inputs block approval, route tradeoffs stay visible, and optional places come after the mandatory transfer. Entry discovery cannot become permission, AI cannot become a safety decision, a seat map cannot become an airline capacity guarantee, and a priced offer cannot become a ticket.

For the HND-to-NRT example, the correct current behavior is to recognize the Tokyo multi-airport relationship, show the three research transfer modes as demo estimates, withhold a live fastest claim, calculate an NRT arrival target only after required timing inputs resolve, and permit an interval activity only after the complete transfer and onward-airport process remain inside the conservative safety policy.

The synchronized credential-free acceptance milestone is complete: clean terminal build, 78 unit tests, 13 UI tests, 37 Worker tests, Worker check, Wrangler dry run, and an ordinary non-test simulator walkthrough all passed or were visually accepted. The generated PDF and provenance artifacts record the same snapshot. Separate credential, account, deployment, and hardware checks are still required before claiming live provider contracts, production authentication from iOS, live delay/capacity/booking adapters, Google OAuth delivery, cross-device alerts, live AI inference, or physical-device AR.